

GPU computing

Simone Petta

A.A. 2024/2025

Contents

1	Introduzione	4
1.1	GP-GPU	5
1.2	Architetture eterogenee	5
1.2.1	Parallelismo delle istruzioni	6
1.2.2	Parallelismo dei dati	6
1.3	Parallelismo di task	6
1.4	Tassonomia di Flynn	6
1.5	GPU Nvidia	7
1.5.1	Cuda core	7
1.5.2	CUDA	7
1.5.3	Hello world in cuda	8
2	Modello di programmazione CUDA	9
2.1	Modelli per sistemi paralleli	10
2.1.1	SIMD	10
2.1.2	MIMD	10
2.1.3	PRAM	10
2.1.4	Parallelizzazione di algoritmi	11
2.2	CPU multi-core e programmazione multi-threading	11
2.2.1	Thread	11
2.3	Programmazione multi-core	12
2.4	Programmazione CUDA	12
2.4.1	Organizzazione dei thread	13
2.4.2	Lancio di un kernel CUDA	14
2.4.3	Kernel concorrenti	15
2.4.4	Restrizioni del kernel	15
2.4.5	Memoria	15
2.5	Modello di esecuzione CUDA	17
2.5.1	Panoramica	17
2.5.2	Warp: architettura SIMT	18
2.5.3	Latency hiding	20
2.5.4	Warp divergence	20
2.5.5	Sincronizzazione	21
2.5.6	Reduction	21
2.5.7	Operazioni atomiche	23

3	Architetture delle GPU	25
3.1	Loop Unrolling	26
3.1.1	Block unrolling	26
3.2	Architetture delle GPU	26
3.2.1	Compute Capability	26
3.2.2	Architettura interna GPU	27
3.2.3	Scheduler di warp	27
3.2.4	Fermi: kernel e memoria	27
3.2.5	Transparent scalability	27
3.2.6	Parallelizzazione dinamica	27
4	Memorie	29
4.1	Registri	30
4.2	Local Memory	30
4.3	Constant Memory	30
4.4	Texture Memory	31
4.5	Shared Memory	31
4.5.1	Organizzazione	31
4.5.2	SMEM runtime	31
4.5.3	Pattern di accesso	31
4.5.4	Conflitto di accesso	32
4.5.5	Configurazione SMEM / cache L1	32
4.5.6	Osservazioni	32
4.5.7	Allocazione statica delle SMEM	33
4.5.8	Allocazione dinamica della SMEM	33
4.5.9	Uso tipico	33
4.5.10	Prodotto matriciale con SMEM	33
4.5.11	Prodotto convolutivo con SMEM	34
4.6	Global Memory	35
4.6.1	Pinned memory	37
4.6.2	Unified Virtual Addressing UVA	37
4.6.3	Pattern di Accesso alla Global Memory	38
5	Ottimizzazione delle prestazioni	41
5.1	Stream e concorrenza	42
5.1.1	CUDA Stream	42
5.1.2	CUDA event	46
5.2	Librerie CUDA	47
5.2.1	Workflow tipico	48
5.2.2	Libreria cuBLAS	48
5.2.3	cuRAND	51
5.2.4	cuFFT	52
6	Numba	54
6.1	Numba	55
6.1.1	Numba for CPU	55
6.1.2	Numba for GPU	56

6.1.3	Gestione della memoria	57
6.1.4	Atomic Operations	59
6.1.5	Streams	59
7	PyTorch	61
7.1	Tensori	62
7.2	Operazioni	63
7.2.1	Spostare sulla GPU	63
7.3	Linear Neural Networks	64
7.3.1	Discesa del gradiente	64
7.3.2	Autograd	66
7.4	Multi Layer Perceptron MLP	67
7.4.1	Modulo <code>nn</code>	67

Chapter 1

Introduzione

L'obiettivo del corso e' di affrontare il problema del high performance computing. Affronteremo anche il tema delle GPU e del loro utilizzo nel calcolo ad alte prestazioni. Inoltre useremo il framework CUDA per sviluppare applicazioni parallele utilizzando il linguaggio CUDA-c che e' un'estensione del linguaggio c. Introduciamo anche alcune librerie di python, andando a focalizzarci su elementi a basso livello.

Il parallel computing e' la necessita' di accelerare le computazioni avvalendosi di un numero molteplice di processori. Quando ho un processo di grandi dimensioni devo essere in grado di dividere il processo in parti e assegnarlo a vari processi. E' fondamentale la progettazione del problema per spezzettarlo in problemi autonomi.

La CPU ha degli aspetti critici, sono decenni che non vediamo crescere il clock (quante operazioni puo' fare al secondo) si e' andati incontro a misure fisiche che non possono essere superate, come la potenza dissipata. Una naturale estensione e' stata aumentare il numero di core, da multi core a many many core dando vita a device diversi, le GPU, che riscono a mantenere la legge di Moore e a fare si che a costi bassi una workstation diventi una macchina ad alte prestazioni. Bisogna dare credito a Nvidia che ha creato le general purpose GPU, che sono delle GPU che possono essere utilizzate per calcoli generali e non solo per il rendering grafico.

Il parallel computing e' indipendenza, comunicazione e scalabilita' dei processi.

Le motivazioni che portano all'utilizzo della GPU, un esempio e' la riflessione, rifrazione e ombra, la cosa che si fa e' avere un calcolo semplice che si fa per ogni pixel, quindi calcoli semplici ripetuti un numero esorbitante di volte. Vengono anche usate per il deeplearning. Perche' le CNN sono di interesse per la GPU? fanno un'operazione basilare da ripetere un numero esorbitante di volte, fanno la convoluzione, prendono un'immagine ed un kernel e scandiscono l'immagine sottostante estraendone matrici della dimensione del kernel e fare la somma. Un altro esempio e' il calcolo matriciale, vengono fatti due prodotti interni tra i vettori riga e colonna delle matrici, questo puo' essere parallelizzato pensando che gruppi di thread distinti possono occuparsi di ogni entry, fare questa cosa in sincrono costerebbe $O(n^3)$.

1.1 GP-GPU

In sistemi eterogenei, le GPU possono essere utilizzate insieme alle CPU per ottenere prestazioni superiori. La chiave per sfruttare al meglio queste architetture eterogenee e' la suddivisione del lavoro tra le diverse unita' di elaborazione, in modo che ogni tipo di processore possa occuparsi delle operazioni per cui e' piu' adatto. E' importante notare che questa architettura e' di tipo master-slave, dove la CPU funge da master e le GPU da slave. Una funzione gpu e' scritta per sfruttare il parallelismo e CUDA ci permette di pensare il sequenziale nonostante poi i calcoli vengano eseguiti in parallelo, l'obiettivo e' aumentare la potenza di calcolo.

1.2 Architetture eterogenee

C'e' la GPU e la CPU, la CPU ha un numero di core. C'e' un bus di cui per la comunicazione tra CPU e GPU.

Quando si crea codice per queste architetture ci si trova con un eseguibile con blocchi di codice che devono essere eseguiti dalla CPU e blocchi dalla GPU.

Le due parti sono destinate a compiti diversi, se i dati sono piccoli e il parallelismo e' basso siamo nel dominio della CPU ed e' quasi inutile fare il parallelismo con la GPU (solo l'overhead del setup iniziale costa di piu') viceversa se i dati sono grandi e il parallelismo e' alto, allora ha senso utilizzare la GPU.

1.2.1 Parallelismo delle istruzioni

Il parallelismo delle istruzioni e' importante (meno di quello dei dati), nel momento in cui il problema puo' essere suddiviso in diverse istruzioni queste vengono eseguite in parallelo. Ci vuole una dotazione hardware adeguata per gestire questo tipo di parallelismo, come ad esempio un numero sufficiente di unità di esecuzione e una gestione della comunicazione tra processi.

1.2.2 Parallelismo dei dati

Noi lavoreremo in una logica sequenziale ma si estende ad una grossa mole di dati. In questo caso, il parallelismo dei dati diventa cruciale, poiché consente di elaborare simultaneamente grandi volumi di informazioni. Utilizzando tecniche di parallelizzazione, possiamo suddividere i dati in blocchi più piccoli e distribuirli su più unità di elaborazione, massimizzando così l'efficienza e riducendo i tempi di calcolo.

1.3 Parallelismo di task

Il fatto di poter avere gruppi di operazioni distinti, ci porta ad un problema non banale di identificazioni di task indipendenti dall'altri, spesso ci si avvale di strumenti come i grafi. Due task sono indipendenti se le operazioni che li compongono non sono dipendenti tra di loro. Questo problema in termini piu' formali e' di colorazione del grado, ogni colore individua un gruppo di nodi indipendenti tra di loro. Non e' un problema banale, perche' e' irrisolvibile NP-hard, quindi ci si accontenta di soluzioni approssimative.

1.4 Tassonomia di Flynn

Si evidenzia i modelli **SISD** single instruction single data dove non c'e' nessun parallelismo e le operazioni vengono eseguite sequenzialmente. In contrapposizione abbiamo il **SIMD** (single instruction multiple data) dove abbiamo un'istruzione che viene eseguita su piu' dati, sono spesso dotate di librerie e hardware consistente che possono svolgere operazioni parallele, in questo caso c'e' l'accesso ad una memoria globale. **MISD** (multiple instruction single data) e' un modello in cui piu' istruzioni vengono eseguite su un singolo dato, mentre il **MIMD** (multiple instruction multiple data) consente l'esecuzione di piu' istruzioni su piu' dati, rappresentando il massimo livello di parallelismo.

Il modello **SIMT** e' interessante e viene introdotto da CUDA, qui ci sono tanti thread che svolgono tante operazioni su thread distinti, deve esserci un sistema book treading. C'e' un'evoluzione del modello SIMD perche' lo cambia con il modello multithreading,

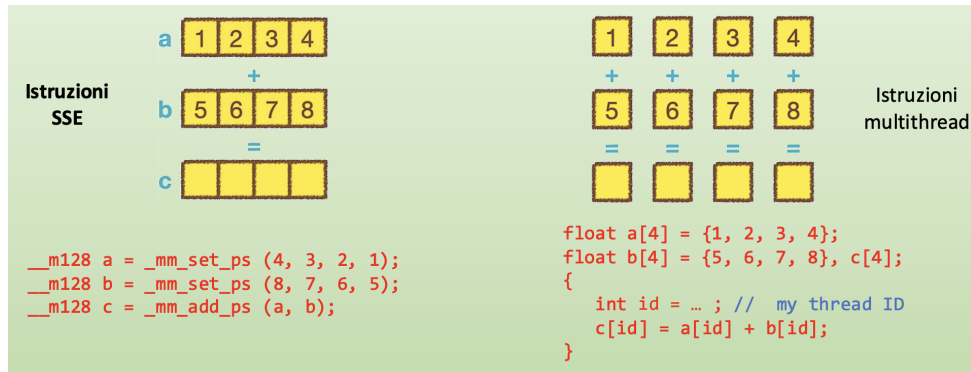


Figure 1.1: Confronto tra SIMD e SIMT

decade il vincolo della singola istruzione, questo e' dato dalla presenza di branch nel codice. Questo modello e' molto utile perche' ci permette di pensare in modo sequenziale anche se complica un po' l'architettura perche' bisogna fare comunicare i thread. Questo e' il modello implementato dai processori GPU.

Nel simt si fissano gli operandi, mentre nel simd no.

1.5 GPU Nvidia

CUDA e' una libreria che permette di fare applicazioni sulle GPU

Ogni scheda e' composta da streaming multi processor, ogni streaming contiene dei core che costituiscono l'architettura della scheda, ci sono gli elementi di memoria shared e global.

1.5.1 Cuda core

Un core ha una logica di processazione vettoriale, piu' dati insieme che possono essere caratterizzati da diverse unita' di calcolo (floating point).

1.5.2 CUDA

E' la piattaforma che ci permette di dare luogo ad un modello di programmazione per le schede Nvidia. Guardando a CUDA nello specifico per progettare delle applicazioni accelerate per le GPU useremo delle librerie gia' accelerate per GPU (come il compilatore).

Ci sono due famiglie di api:

- CUDA Runtime
- CUDA Driver: sono a basso livello e sono piu' difficili da utilizzare perche' non astraggono determinate cose

Programma CUDA

Un programma cuda e' composto da due parti:

- codice host eseguito su CPU
- codice device eseguito su GPU

Il compilatore separa il codice host dal codice device in fase di compilazione, generando un file eseguibile che contiene entrambe le parti.

1.5.3 Hello world in cuda

Per scrivere programmi cuda va modificata la sintassi del c, si scrivono delle funzioni che sono destinate al kernel cuda:

```
1  __global__ void hello_world() {
2      printf("Hello, World from GPU!\n");
3  }
```

Per invocarla va utilizzata questa sintassi in cui specifichiamo il numero di blocchi e il numero di thread per ogni blocco:

```
1  hello_world<<<num_blocks, threads_per_block>>>();
```

Il risultato finale e':

```
1  __global__ void hello_world() {
2      printf("Hello, World from GPU!\n");
3  }
4
5  int main(void) {
6      hello_world<<<1, 10>>>();
7      cudaDeviceReset();
8      return 0;
9  }
```

In questa funzione stamperemo 10 volte "Hello, World from GPU!".

La funzione `cudaDeviceReset` serve a ripristinare lo stato del dispositivo GPU. Viene utilizzata per liberare le risorse allocate sulla GPU e riportare il dispositivo a uno stato pulito. È buona norma chiamare questa funzione alla fine di un programma CUDA per garantire che tutte le risorse siano correttamente rilasciate.

Chapter 2

Modello di programmazione CUDA

2.1 Modelli per sistemi paralleli

E' qualcosa di complesso che richiede una parte HW e software, bisogna necessariamente fare delle astrazioni perche' non ci potremo occupare di tutti i dettagli. Abbiamo questa astrazione:

- Livello macchina: assembly
- Modello architetturale: SIMD o MIMD, si prevedono anche processi in cui ci sono tante unita' di calcolo che interagiscono tra di loro, gestione multi processi e multi thread
- Modello computazionale: e' un modello formale che consente di descrivere la macchina astraendo dai modelli sottostanti ed e' fondamentale per la creazione degli algoritmi. Processore, memoria, scambio dati con un BUS. E' inerentemente sequenziale ma e' utile per creare l'algoritmo data la size dei dati. Nell'ambito del parallelismo passiamo dal modello RAM al PRAM (modello RAM per architetture parallele)
- Modello di programmazione parallela: e' il modo con cui gestiamo il sistema di calcolo parallelo esprimendo algoritmi con un linguaggio che espone il parallelismo dell'architettura sottostante per essere eseguito da un linguaggio che ha in se elementi che permettono di usare il parallelismo. Vi sono aspetti di comunicazione per gestire il multi threading, si affronta quindi la suddivisione del compito in processi e thread. Fino ad arrivare a problemi legati all'uso o della gestione di spazi di indirizzamento condivisi.

2.1.1 SIMD

Si parla di unita' molteplici che lavorano sui dati, laddove i dati permettono di sviluppare meccanismi paralleli. E' un mondo vasto che si puo' vedere anche nelle architetture comuni dove si ha la divisione di computazione in diverse ALU specializzate, una che lavora su un singolo registro ed una che lavora vettorialmente ed e' quindi in grado di lavorare su piu' dati contemporaneamente. Nelle GPU parleremo di thread che lavorano simultaneamente in modalita' SIMD.

2.1.2 MIMD

E' un'altro dei modelli molto diffusi paralleli in cui si hanno le istruzioni di tanti dati simultaneamente che vengono eseguite da un numero di processori veramente grandi.

2.1.3 PRAM

Il modello PRAM e' il modello di calcolo parallelo piu' semplice, in questo caso abbiamo una memoria condivisa, i processori lavorano in maniera SIMD ovvero fanno la stessa istruzione, esistono anche processori inattivi. Ci da quanti passi ci servono per risolvere l'algoritmo parallelo, anche in questo caso potremmo fare studio di complessita' per risolvere il problema parallelo.

2.1.4 Parallelizzazione di algoritmi

Quando vorremo vedere la complessita' computazionale degli algoritmi dovremo fare delle astrazioni ed usare qualcosa simile al PRAM. Nella pratica si partira' da un algoritmo sequenziale, dovremo vedere come dividere la computazione in parallelo per capire come le varie parti vengano attribuite ai vari thread, tendenzialmente la partizione logica dei task la faremo noi, la parte dello scheduling se ne occupa il sistema ma e' bene conoscerla perche' ad esempio in CUDA i processi di scheduling sono fortemente condizionanti dalla potenza del sistema e dovremo adeguare gli algoritmi sulla base dei meccanismi di scheduling. Quello che vorremo ottenere e' una massima occupancy delle unita' di calcolo. Dipendentemente dal modello di memoria, un task puo' accedere a memoria condivisa o usare tecniche di message passing.

2.2 CPU multi-core e programmazione multi-threading

Un programma in esecuzione e' un processo che viene allocato nel OS e ha una serie di risorse che ha, convive con altri processi nel sistema ed e' una cosa abbastanza pesante, mentre i thread che compongono un processo sono dei sotto-processi ma piu' leggeri. Un processo in genere consiste di tanti thread. In genere vengono creati da delle operazioni di sistema come le fork in UNIX e vengono terminati dall'OS. I processi hanno il loro contesto e vengono eseguiti sequenzialmente secondo un flusso, anche i thread eseguono le cose sequenzialmente ma essendo molteplici possono essere paralleli. Il thread e' un singolo flusso di istruzione che si trova in un processo che lo scheduler si occupa di eseguire concorrentemente ad altri threads.

2.2.1 Thread

Ha un ciclo di vita, viene generato e viene ucciso, il processo chiude quando tutti i thread hanno terminato la loro esecuzione:

- New: viene generato, chi lo ha generato sa chi e'
- Executable: il thread e' pronto per essere eseguito quando ha tutte le risorse necessarie
- Running: il thread sta attualmente eseguendo le sue istruzioni
- Waiting: il thread sta aspettando che una risorsa diventi disponibile
- Finished: il thread ha completato la sua esecuzione e viene rimosso

I vantaggi sono:

- Si possono avere tanti flussi di esecuzioni che possono avvantaggiarsi di sistemi multi-core
- Visibilita' dei dati globale
- Semplicita' di gestione eventi asincroni come l'io

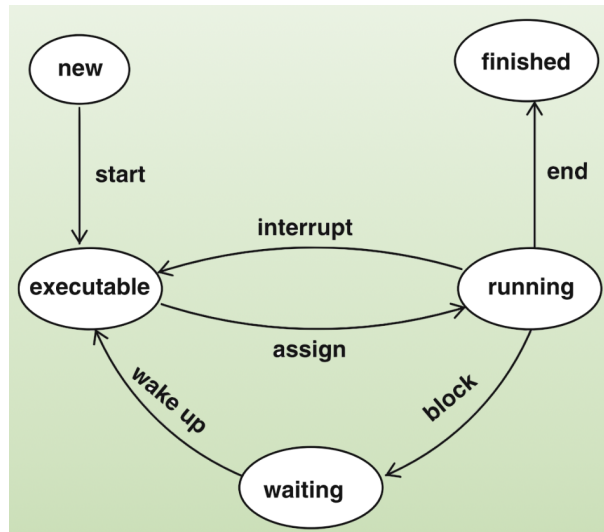


Figure 2.1: Stati di un thread

- Velocita' di context switching

Le difficolta' sono la concorrenza e il problema di avere uno spazio privato.

2.3 Programmazione multi-core

Noi vedremo la programmazione multi-thread su architetture multi-core. L'applicazione deve essere progettata considerando diversi fattori del sistema di calcolo parallelo multi-core (e many core come le GPU), bisogna trovare di task, bilanciarli, suddividere i dati sui vari task (il problema e i dati lo devono consentire) e farlo in modo che tutti i task possano lavorare su queste porzioni di dati indipendentemente. Ci sono quindi tanti aspetti da tenere in considerazione sulla dipendenza e indipendenza dei dati. Test e debugging sono anchessi importanti quando si sviluppano gli algoritmi in ambito parallelo perche' ci sono tanti flussi di esecuzione.

2.4 Programmazione CUDA

Pensare in parallelo significa avere chiaro quali feature la GPU espone al programmatore. Si lavora come su pthread o OpenMP. Si scrive una porzione di CUDA C (semplice estensione di C) per l'esecuzione sequenziale e lo si estende a migliaia di thread (permette di pensare 'ancora' in sequenziale)

- I dati stanno su host
- allocare spazio su GPU
- copiare i dati da host a GPU
- allocare memoria per l'output

- lanciare il kernel che elabora dati in ingresso e li mette in output, mette una copia su memoria condivisa
- cancellare le memorie allocate

2.4.1 Organizzazione dei thread

Cuda presenta una gerarchia astratta di thread che si distribuisce su due livelli:

- grid: una griglia ordinata di blocchi
- block: un insieme di thread che possono cooperare tra loro

Questi possono avere dimensioni 1D, 2D o 3D. Tutto questo fa 9 combinazioni tra grid e block.

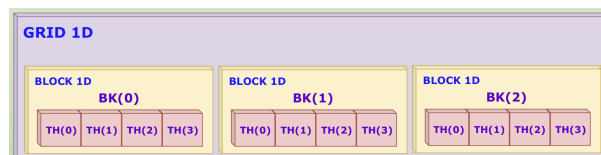


Figure 2.2: Organizzazione dei thread in CUDA

Ci sono delle limitazioni, un blocco puo' contenere al massimo 1024 thread.

Il blocco di thread e' molto importante, dal punto di vista semantico ha un significato, e' un gruppo di thread che possono cooperare tra loro tramite:

- block-local synchronization, sincronizzazione: cooperare per uno stesso compito avvantaggiandosi da operazioni fatte da altri thread
- block-local communication, comunicazione tramite la shared memory: e' una memoria cache che quindi ha tempi di accesso di molto ridotti

I thread vengono identificati univocamente tramite l'id di blocco e l'id di thread. blockIdx e threadIdx sono specificati in variabili globali, un kernel a runtime ha accesso a queste informazioni che vengono assegnate dinamicamente, hanno tre valori x, y, z di tipo uint32.

```

1
2 #include <stdio.h>
3
4 __global__ void checkIndex(void) {
5     printf("threadIdx: (%d, %d, %d) blockIdx: (%d, %d, %d) "
6           "blockDim: (%d, %d, %d) gridDim: (%d, %d, %d)\n",
7           threadIdx.x, threadIdx.y, threadIdx.z,
8           blockIdx.x, blockIdx.y, blockIdx.z,
9           blockDim.x, blockDim.y, blockDim.z,
10          gridDim.x, gridDim.y, gridDim.z);
11 }
12
13 /*
14 * MAIN

```

```

15 */
16 int main(int argc, char **argv) {
17
18     // grid and block definition
19     dim3 block(4);
20     dim3 grid(3);
21
22     // Print from host
23     printf("Print from host:\n");
24     printf("grid.x = %d\t grid.y = %d\t grid.z = %d\n", grid.x, grid.y,
25           grid.z);
26     printf("block.x = %d\t block.y = %d\t block.z %d\n\n", block.x,
27           block.y, block.z);
28
29     // Print from device
30     printf("Print from device:\n");
31     checkIndex<<<grid, block>>>();
32
33     // reset device
34     cudaDeviceReset();
35     return(0);
36 }

```

Abbiamo accesso alle dimensioni della griglia e del blocco tramite le variabili `blockDim` e `gridDim`, che sono anchessi di tipo `uint32`. Anche in questo caso abbiamo tre componenti `x`, `y`, `z`.

L'indice univoco del thread nei blocchi si calcola diversamente rispetto alle dimensioni del blocco:

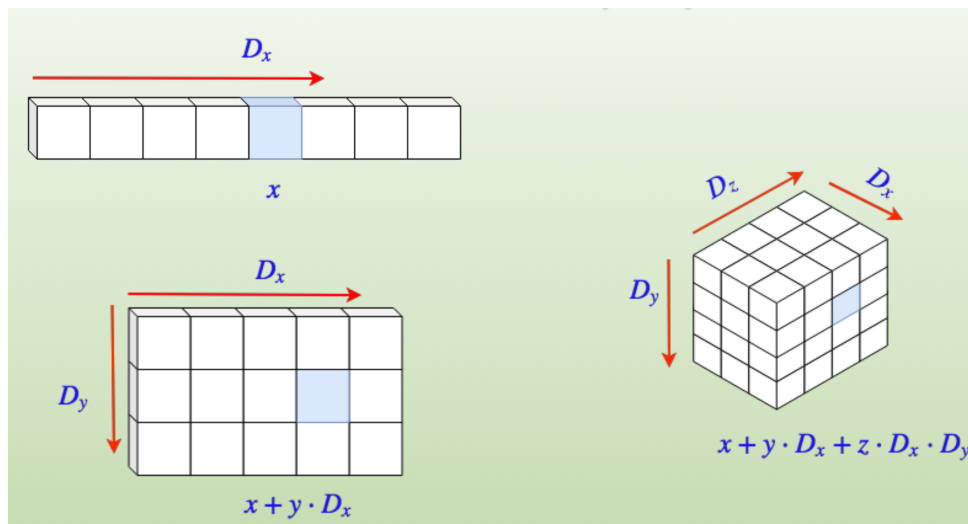


Figure 2.3: Indice univoco dei thread nei blocchi

2.4.2 Lancio di un kernel CUDA

Quando prendo una funzione e la lancio con un kernel significa che prendo quella funzione e la lancio sulla GPU, i parametri di configurazione di esecuzione sono fatti in questo modo:

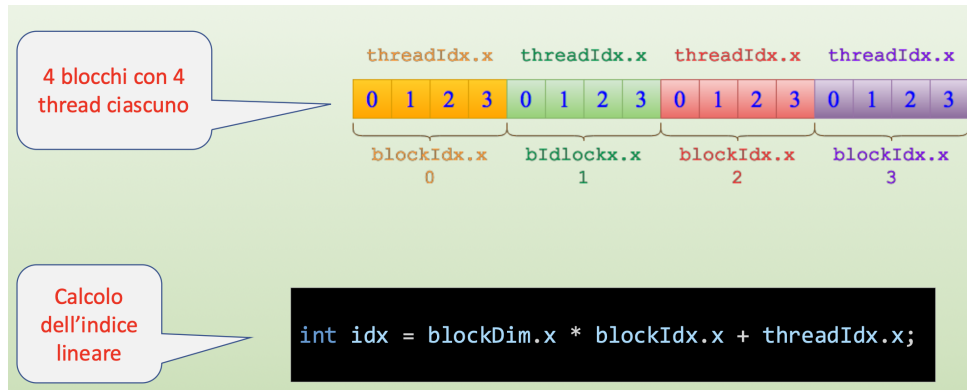


Figure 2.4: Organizzazione della griglia 1D e coordinate 1D

```
1 kernel<<<gridDim, blockDim>>>(args...);
```

Il primo valore `gridDim` specifica la dimensione della griglia di blocchi, mentre il secondo valore `blockDim` specifica la dimensione di ciascun blocco di thread. Gli argomenti `args...` rappresentano i parametri da passare al kernel.

2.4.3 Kernel concorrenti

Potrebbe verificarsi che numerosi kernel vengano lanciati sullo stesso host (dallo stesso processo o da processi diversi), potrei trovarmi nella situazione in cui ho diversi blocchi concorrenti sullo stesso device.

Esiste l'api `cudaDeviceSynchronize` che permette di sincronizzare l'esecuzione dei kernel, significa che la cpu si sincronizza con tutti i kernel, praticamente il lancio del kernel diventa bloccante.

2.4.4 Restrizioni del kernel

- Accede alla memoria globale
- Restituisce void
- non supporta numero variabile di argomenti, devono essere specificati
- non supporta le variabili statiche
- ha un comportamento asincrono rispetto al chiamante

2.4.5 Memoria

E' praticamente una trasposizione 1 a 1 della memoria in C, le funzioni servono per allocare memoria sul device in global memory (ricordasi che i thread accedono tutti alla global memory):

- `cudaMalloc`: alloca memoria sul device

- cudaFree: dealloca memoria sul device
- cudaMemcpy: copia dati tra host e device
- cudaMemcpySet: imposta un valore nella memoria del device

Il trasferimento e l'allocazione di memoria sono operazioni sincrone, l'host si ferma in attesa del completamento di queste operazioni.

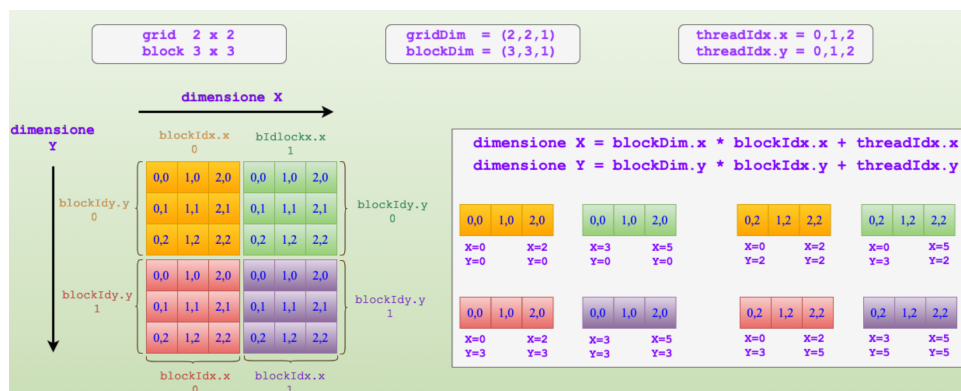
cudaMalloc ritorna un doppio puntatore, che e' quello che viene passato ai kernel per puntare correttamente allo spazio di indirizzamento del device (memoria globale), la size e' in byte, il puntatore e' di tipo void:

```
1 cudaError_t cudaMalloc(void** devPtr, size_t size);
```

restituisce un codice di errore. La cudaFree e' quella che libera la memoria.

I puntatori su host e device sono diversi, non e' possibile fare assegnamenti tra uno e l'altro invece bisogna usare la cudaMemcpy.

Non c'e' sempre un mapping 1-1 tra i puntatori host e device, guardiamo ora il mapping tra griglia 2d e coordinate 2d. Se prendiamo una griglia di blocchi 2x2 mostriamo gli indici dei blocchi e dei thread all'interno, mostriamo che poiche' i tre campi vengono definiti quando definiamo le griglie dobbiamo capire come usare x e y, il campo x viene considerato come campo che varia sulle colonne della matrice per converso la y indica le righe della matrice e si muove in verticale pensando che l'origine e' in alto a sinistra. Posso creare un doppio ordine per x e y di indici e le coppie ordinate mi indicizzano tutte le entry della matrice:



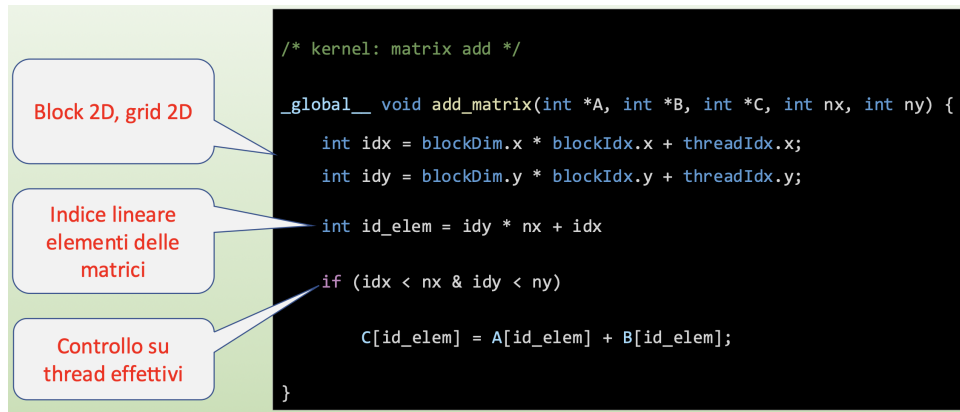
Abbiamo quindi prodotto delle coordinate 2d (x,y) per ogni thread all'interno della griglia 2D e a questo punto e' sufficiente spostarsi nella colonna e nella riga:

```
1 int ix = blockDim.x * blockIdx.x + threadIdx.x;
2 int iy = blockDim.y * blockIdx.y + threadIdx.y;
```

A questo punto l'indice linearizzato lo ricaviamo in questo modo:

```
1 int id = iy * nx + ix;
```

Se si deve trattare una matrice che rappresenta l'immagine, si possono organizzare blocchi di dimensione fissata (16x16) e calcolare la griglia di conseguenza (5x4).



2.5 Modello di esecuzione CUDA

Vogliamo vedere cosa succede a questa miriade di thread che vengono eseguiti nelle GPU, come si arriva ai core. Lo faremo in modo astratto dall'architettura, senza entrare nei dettagli specifici delle implementazioni hardware.

2.5.1 Panoramica

Quando abbiamo un thread che ha le sue risorse private (dei registri assegnati ad uno spazio locale) ed e' a sua volta in grado di chiamare delle funzioni. I thread vengono ordinati in blocchi, il blocco viene attribuito ad uno streaming multi-processor e devono essere eseguiti in parallelo su una risorsa. Vi e' l'uso e lo scambio di memorie veloci, la shared memory per la comunicazione inter-thread. Una griglia (grid) e' un array di thread block che eseguono tutti lo stesso kernel, legge e scrive in global memory e sincronizza le chiamate di kernel tra loro dipendenti. L'architettura della GPU e' disegnata come un array scalabile di streaming multiprocessors, il parallelismo hardware della GPU e' ottenuto replicando questo elemento base. Gli elementi di uno streaming multiprocessor tipo sono:

- CUDA core
- Memory: global - shared - texture - constant
- caches
- registri per la memoria locale
- unita' load/store per i/o
- unita' per funzioni specializzate, come funzioni algebrice sin, cos, exp
- scheduler dei wrap

Gli vengono assegnati i blocchi e lui e' in grado di gestire i blocchi attraverso lo scheduler.

Mapping logico - fisico dei thread

In questo panorama dal punto di vista logico si ha un mapping tra thread e core (ogni thread ha un core) un blocco di thread viene eseguito su un streaming multiprocessor e la griglia viene mappata sul device.

Esecuzione

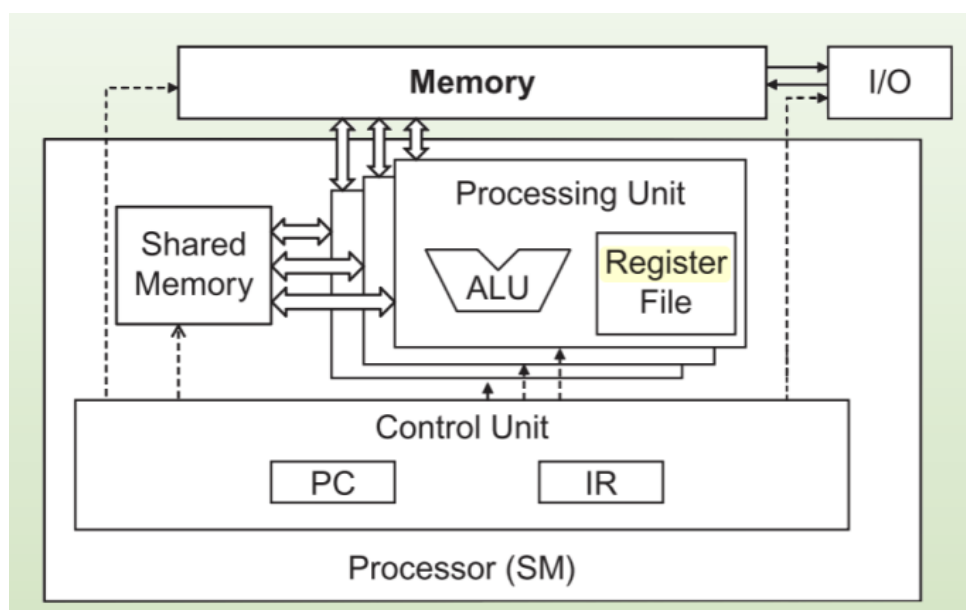
- mapping gerarchia thread in gerarchia processori sulla GPU
- GPU esegue uno o piu' kernel
- streaming multiprocessor gestisce i blocchi di thread
- un thread block e' schedulato solo su un SM
- I core eseguono i thread
- Gli SM eseguono i thread a gruppi di 32 thread chiamati warp

2.5.2 Warp: architettura SIMT

Gli warp sono gruppi di 32 thread (con ID consecutivi) che messi insieme agli altri warp costituiscono un blocco. Ha una semantica che richiama al modello SIMD perche' idealmente si vorrebbe che i thread nel warp eseguissero simultaneamente la stessa istruzione. Questa cosa e' un problema perche' ogni thread ha un flusso a se, quindi si ricorre al modello SIMT questo implica perdita di efficienza. Ogni thread ha il suo program counter e register state. Ogni thread puo' eseguire cammini distinti di esecuzione delle istruzioni. I thread che compongono un warp iniziano assieme allo stesso indirizzo del programma. 32 e' un numero magico, ha origine dall'HW ed e' fondamentale nella programmazione CUDA per la scalabilita' e trasparenza. E' l'unita' minima di esecuzione che permette grande efficienza nell'uso della GPU. Ha un forte impatto sulle prestazioni degli algoritmi sviluppati. Concettualmente ha un comportamento modello SIMD ma nella pratica assume un modello SIMT.

HW multi-threading

Questi sono gli elementi HW di un ambiente multi-threading:



Registri

I registri sono usati per le variabili automatiche scalari e le coordinate dei thread:

```
1  __global__ void matrix_prod(float *a, float *b, float *c) {
2      int Row = blockIdx.y * blockDim.y + threadIdx.y;
3      int Col = blockIdx.x * blockDim.x + threadIdx.x;
4      if (Row < N && Col < N) {
5          float sum = 0.0f;
6          for (int k = 0; k < N; k++) {
7              sum += a[Row * N + k] * b[k * N + Col];
8          }
9          c[Row * N + Col] = sum;
10     }
11 }
```

Logica warp

Dal punto di vista logico i warp mantengono la consistenza ma il blocco la perde un po', lo vediamo come blocco di thread ma quando viene passato alla macchina e diventa attivo questo viene ripartito in warp (la macchina vede una collezione di 32 thread) e poi lo passano al multiprocessore che si occupa dei warp che gli competono, questo accade indipendentemente dalla dimensione di griglia che abbiamo fissato. I warp condizionano come noi creiamo il kernel, ad esempio un kernel di 40 blocchi non ha molto senso, dato che dobbiamo eseguire gruppi di 32 thread alla volta e' meglio avere multipli di 32 (gli altri rimarrebbero inattivi e si mangerebbero risorse inutili). Uno warp puo' essere attivo o disattivo, e' attivo quando le risorse di computazione gli vengono assegnate, quando e' attivo si puo' trovare in diversi stati:

- selezionato: in esecuzione su un dato path (preso in carico dallo scheduler di warp)
- bloccato: non pronto per l'esecuzione
- candidato: eleggibile se tutti e 32 i core sono liberi e tutti gli argomenti della prossima istruzione sono disponibili

Scheduling dei blocchi

Come funziona lo scheduling dei blocchi, ogni streaming multiprocessor riceve un numero di blocchi (questo varia a seconda di quanti SM ho) questo per non lasciare IDLE gli SM.

Ripartizione risorse

Le risorse vengono ripartite a seconda della griglia che andiamo a definire e nel momento in cui vengono ripartite non c'e' context switch ed e' per questo che cio' che puo' essere attivo e' limitato. Esistono limiti imposti dall'HW, come ad esempio il numero di blocchi che possono essere attivi contemporaneamente su un SM, il numero di thread per blocco, il numero di registri per thread, la dimensione della shared memory per blocco, la dimensione della global memory per blocco.

L'occupancy e' il tasso tra warp attivi e numero massimo di warp per SM:

$$\text{occupancy} = \frac{\text{warp attivi}}{\text{numero massimo di warp per SM}}$$

Esiste un flag di uno strumento di profilazione per calcolare l'occupancy:

```
1 nvprof --metrics achieved_occupancy ./Application
```

2.5.3 Latency hiding

Il grado di parallelismo a livello di thread utile a massimizzare l'utilizzo delle unita' funzionali di un SM dipende dal numero di warp residenti e attivi nel tempo. La latenza e' definita come il numero di cicli necessari al completamento dell'istruzione. Per massimizzare il throughput occorre che lo scheduler abbia sempre warp eleggibili ad ogni ciclo di clock. Si ha cosi' latency hiding quando i warp in attesa di esecuzione possono essere utilizzati per mascherare la latenza delle operazioni in corso. Classificazione dei tipi di istruzione che inducono latenza:

- Istruzioni aritmetiche: tempo necessario per la terminazione dell'operazione
- Istruzioni di memoria: tempo necessario al dato per giungere a destinazione

2.5.4 Warp divergence

La divergenza e' un altro nemico insieme alla latenza, si esplicita in modo chiaro, ci sono dei branch all'interno del codice (if else), semplicemente di fronte ad un thread si ha un ritardo sistematico e perdiamo la natura del parallelismo di un warp. Fa cadere il modello SIMD e diventa SIMT.

```
1  __global__ void pari_dispari_1(int *c) {
2      int tid = blockIdx.x * blockDim.x + threadIdx.x;
3      int a = 0, b = 0;
4
5      if (tid % 2 == 0) {
6          a = 2;
7      } else {
8          b = 1;
9      }
10
11     c[tid] = a + b;
12 }
```

Il compilatore fa dell'ottimizzazione per la divergenza (dove puo' ovviamente), aggiunge dei predicati alle istruzioni che verificano il vero o falso, calcolano quindi il predicato logico e le istruzioni del predicato e a questo punto i thread calcolano i predicati e non si sequenzializzano.

```
1  if (x < 0) {
2      z = x - 2
3  }
4  else if (x >= 0) {
```

```

5      z = x + 2
6  }
7
8
9  p = (x<0)
10 p: z = x - 2
11 !p: z = x + 2

```

2.5.5 Sincronizzazione

E' un altro aspetto fondamentale a livello di blocco, i thread possono seguire strade diverse e possono avere tempi diversi. Possiamo arrivare ad uno stadio in cui tutti dei thread hanno finito la loro esecuzione, e' importante quindi impostare delle barriere di sincronizzazione in cui tutti i thread si aspettano, questo per evitare delle race condition e per fare in modo di usare il risultato di altri thread. Si puo' fare a livello di sistema, aspettiamo che venga completato un dato task su entrambi host e device:

```

1  cudaError_t cudaDeviceSynchronize(void);

```

ma anche a livello di blocco, si tratta di utilizzare le primitive di sincronizzazione fornite da CUDA, come `__syncthreads()`, che permette di sincronizzare i thread all'interno di un blocco.

```

1  __device__ void __syncthreads(void);

```

Ma anche a livello di warp, attendiamo che tutti i thread in un warp raggiungano lo stesso punto di esecuzione:

```

1  __device__ void __syncwarp(mask);

```

Deadlock

Attenzione che sincronizzazione e divergenza possono portare ad un deadlock, quindi è importante progettare attentamente la logica del kernel per evitare situazioni di stallo.

```

1  if (threadIdx.x < 16){
2      F1();
3      __syncthreads();
4  }
5  else if (threadIdx.x >= 16) {
6      F2();
7      __syncthreads();
8  }

```

La prima meta' dei thread aspetta la seconda meta' per completare la propria esecuzione, creando una situazione di stallo.

2.5.6 Reduction

La reduction e' un operazione molto comune che e' la somma degli elementi di array di grandi dimensioni, se lo vogliamo fare in parallelo non e' cosi' scontata, possiamo fare

questa cosa quando l'operatore soddisfa le proprietà commutativa e associativa, questo significa che possiamo fare questa operazione riordinando i dati come vogliamo

Il caso sequenziale è banale:

```
1 float sum = 0.0f;
2 for (int i = 0; i < N; i++) {
3     sum += array[i];
4 }
```

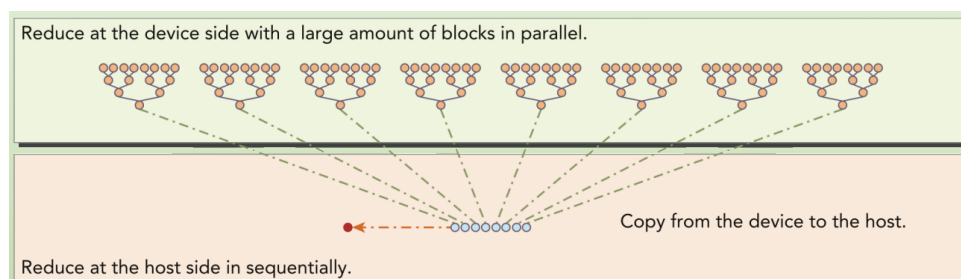
Il caso parallelo è più complesso, dobbiamo dividere i dati in parti uguali, ogni thread calcola la somma della sua parte e poi si fa una somma finale. L'idea quindi è di fare somme parziali memorizzate in-place nel vettore stesso, sommiamoci gli elementi di un livello sulla metà degli elementi del livello sottostante, così facendo teniamo un albero di log n (dove n sono i dati che devono essere sommati). Conviene ricondursi a tecniche ricorsive:

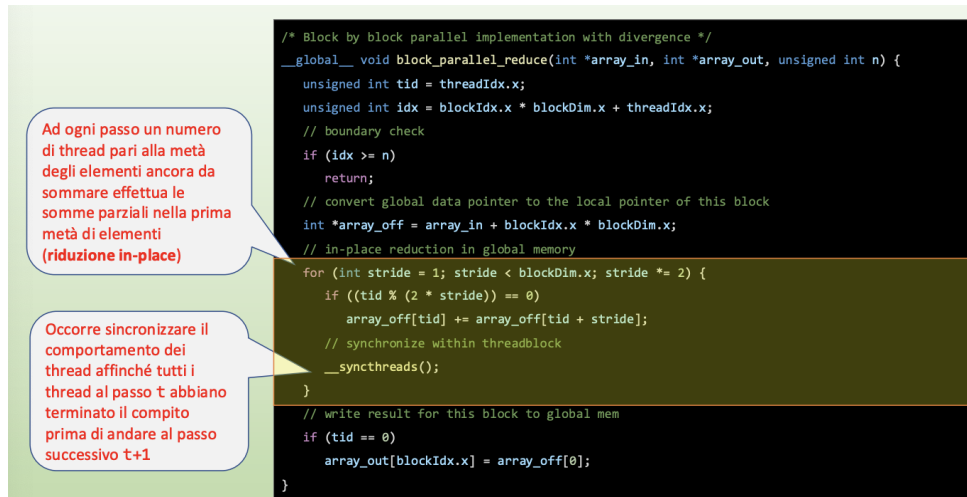
```
1 int recursiveReduce(int *data, int const size) {
2     //terminazione
3     if (size == 1) return data[0];
4
5     //rinnova lo stride
6     int const stride = size / 2;
7
8     //riduzione in loco
9     for (int i = 0; i < stride; i++) {
10         data[i] += data[i + stride];
11     }
12
13     //chiamata ricorsiva
14     return recursiveReduce(data, stride);
15 }
```

Da qui possiamo pensare ad una strategia ricorsiva:

- Ad ogni passo un numero di thread pari alla metà degli elementi dell'array effettua le somme parziali nella prima metà degli elementi, riduzione
- il numero di thread attivi si dimezza ad ogni passo rinnovare lo stride
- occorre sincronizzare il comportamento dei thread affinché tutti i thread al passo t abbiano terminato il compito prima di andare al passo successivo $t + 1$ (analogo alla chiamata ricorsiva)

Dobbiamo pensare che i dati li dividiamo in blocchi, e poi combinare i risultati parziali usciti dai blocchi:





Questa soluzione ha il problema della divergenza dei warp, l'if in cui controlliamo che l'indice del thread sia pari, possiamo però eliminare l'if:

```

1  for (int stride = 1; stride < blockDim.x; stride *= 2) {
2      //convert tid into local array index
3      int index = 2* stride * tid;
4      if(index < blockDim.x) {
5          idata[index] += idata[index + stride];
6          //synchronize within threadblock
7          __syncthreads();
8      }

```

2.5.7 Operazioni atomiche

Poniamo un problema per porre il pretesto di usare delle operazioni atomiche che di perse' garantiscono l'accesso corretto ed univoco ai dati anche se piu' thread accedono allo stesso dato. L'istogramma di un'immagine e' l'intensita' del colore RGB in un'immagine. Per calcolarlo ci serve avere operazioni atomiche perche' dobbiamo incrementare l'array di intensita' RGB accedendo in modo concorrente agli stessi pixel con piu' thread.

Le operazioni atomiche in CUDA eseguono solo operazioni matematiche ma senza interruzione da parte di altri thread questo perche' sono tradotte in una singola istruzione, sono definite da cuda:

```

1  int atomicAdd(int *address, int val);

```

Le operazioni basilari sono:

- Matematiche: add, subtract, maximum, minimum, increment, decrement
- Bitwise: AND, bitwise OR, bitwise XOR
- Swap: scambiano valore in memoria con uno nuovo

L'idea per calcolare l'istogramma e' semplice, si alloca una struttura dati (un array lungo $3 * 256$) per mantenere i valori dell'istogramma e , per ogni pixel dell'immagine si effettua una `atomicAdd()` per incrementare il valore della frequenza dei colori presenti nel pixel.


```

1  /*
2  * Kernel 1D that computes histogram on GPU
3  */
4  __global__ void ppm_histGPU(PPM ppm, int *histogram) {
5
6      uint x = blockIdx.x * blockDim.x + threadIdx.x;
7
8      // pixel out of range
9      if (x >= ppm.width * ppm.height)
10         return;
11
12     // colors of the pixel
13     color R = ppm.image[3 * x];
14     color G = ppm.image[3 * x + 1];
15     color B = ppm.image[3 * x + 2];
16
17     // use atomic
18     atomicAdd(&histogram[R], 1);
19     atomicAdd(&histogram[G+256], 1);
20     atomicAdd(&histogram[B+512], 1);
21 }

```

Prodotto matriciale

Come facciamo ad ottenere il prodotto tra matrici sfruttando la blocchettizzazione. Quello che dobbiamo fare e' definire una griglia definendo il mapping tra indici di thread e delle matrici. Da notare che le dimensioni rilevanti sono 2, numero di righe n della matrice A e il numero m di colonne della matrice B .

Chapter 3

Architetture delle GPU

3.1 Loop Unrolling

L'idea è di ottimizzare i loop ripetendo le istruzioni nel corpo del loop. Invece di eseguire un'iterazione alla volta, il compilatore o il programmatore espande il loop per eseguire più iterazioni in un singolo passaggio. Questo riduce l'overhead del controllo del loop e può migliorare l'uso della pipeline della CPU o della GPU. Come esempio vediamo il for della parallel reduction vista in precedenza:

```
1   for (int stride = blockDim.x / 2; stride > 0; stride /= 2) {
2       if (threadIdx.x < stride) {
3           thisBlock[threadIdx.x] += thisBlock[threadIdx.x + stride];
4       }
5       __syncthreads();
6   }
```

Per farlo è possibile aggiungere un descrittore volatile che implica che ogni volta che l'istruzione viene eseguita va direttamente alla shared memory, evitando ottimizzazioni indesiderate da parte del compilatore passando per la cache:

```
1   if (tid < 32) {
2       volatile int *vmem = thisBlock;
3       vmem[tid] += vmem[tid + 32];
4       vmem[tid] += vmem[tid + 16];
5       vmem[tid] += vmem[tid + 8];
6       vmem[tid] += vmem[tid + 4];
7       vmem[tid] += vmem[tid + 2];
8       vmem[tid] += vmem[tid + 1];
9   }
```

32 non è un numero casuale ma stiamo facendo l'unrolling a livello di warp:

3.1.1 Block unrolling

Possiamo estendere l'idea di loop unrolling anche a livello di block. In questo caso, invece di unrollare le operazioni all'interno di un singolo warp, possiamo unrollare le operazioni su più thread all'interno di un block. Questo approccio può portare a un utilizzo più efficiente delle risorse della GPU e a una riduzione del numero di sincronizzazioni necessarie.

3.2 Architetture delle GPU

Il PCI Express sono i bus che permettono di unire CPU e GPU ed è normalmente il collo di bottiglia del sistema. Ci sono dei controller da entrambe le parti che permettono di parlare con il BUS, all'interno di CPU e GPU ci sono bus per comunicazione interna molto più veloci.

Il PCI Express BUS parla con dei controller che a loro volta si interfacciano con il livello di cache più basso dell'architettura.

3.2.1 Compute Capability

È il termine usato per descrivere la versione hw degli acceleratori GPU che appartengono alle famiglie di dispositivi e che hanno una data architettura interna. È un parametro che

va passato al compilatore, dobbiamo tenere in considerazione questi cambiamenti generazionali delle GPU per scrivere degli algoritmi efficienti per l'architettura che abbiamo a disposizione. Non c'è grande compatibilità tra versioni.

3.2.2 Architettura interna GPU

Ci sono i vari streaming multi processors che comunicano usando una shared memory che è una memoria cache, poi ci sono i vari bus e controllori che parlano con la CPU.

Host interface

È un controller interno alla GPU che si interfaccia verso bus PCIe. Consente alla GPU di trasferire dati da e per la CPU. Ha due tipi di memorie cache: una L1 interna al SM e una L2 condivisa tra SM. Ogni cache L1 è parallela e combina attività con le unità load/store. Un memory controller dedicato trasporta dati dentro e fuori dalla GDDR5 global memory.

Giga-thread scheduler

Si occupa di assegnare ai vari SM disponibili i blocchi del kernel con una politica di round robin. L'assegnamento di blocchi a SM è un'attività veloce, l'esecuzione molto più lenta. Le variabili gridDim, blockIdx e blockDim sono passate dal giga thread scheduler ad ogni core della GPU che eseguirà il corrispondente blocco.

3.2.3 Scheduler di warp

Ogni blocco è assegnato a un SM. Ogni SM contiene 2 warp scheduler e 2 instruction dispatch unit. I due scheduler selezionano 2 warp da eseguire in parallelo e distribuiscono un'istruzione a ognuno dei 16 core o alle 16 unità o alle 4 FPU. L'architettura Fermi può trattare simultaneamente 8 blocchi = 48 warp per SM per un totale di 1536 thread alla volta residenti su un singolo SM.

3.2.4 Fermi: kernel e memoria

La memoria è fissa e può essere divisa in...

3.2.5 Transparent scalability

È la capacità di eseguire lo stesso codice applicativo su diverse architetture, numero variabile di compute core e di SM.

3.2.6 Parallelizzazione dinamica

Potremmo voler lanciare un kernel da un kernel, il kernel padre può consumare l'output prodotto dal figlio tutto senza la CPU:

```
1  __global__ void kernel_figlio() {  
2      // Codice del kernel figlio  
3  }  
4  
5  __global__ void kernel_padre() {  
6      // Codice del kernel padre  
7      kernel_figlio<<<1, 1>>>();  
8  }
```

Bilanciare il lavoro

Se mi accorgo che un blocco di dati e' molto grande e richiede tanto tempo potrei dividere questo blocco in altri blocchi e dividere il lavoro con altri thread. Questo e' utile per evitare che un singolo blocco di dati rallenti l'intero processo di calcolo.

Chapter 4

Memorie

Esiste una gerarchia di memoria, partono da memorie molto veloci come i registri, le cache arriviamo poi alla main memory e alla disk memory, sempre piu' lente e sempre piu' grandi.

Ci sono i principi di localita' e gerarchia di memorie. Il principio di localita' spaziale afferma che i programmi tendono ad accedere a una porzione ristretta di memoria in un dato momento, se accedo ad i e' molto probabile che acceda ad un indirizzo li' vicino (Δi), il che rende vantaggioso mantenere i dati frequentemente utilizzati nelle memorie piu' veloci. La gerarchia di memorie sfrutta questo principio organizzando le memorie in livelli, con memorie piu' veloci e costose in alto e memorie piu' lente e economiche in basso. Mentre la localita' temporale si riferisce alla tendenza ad eseguire la stessa istruzione piu' volte in un breve intervallo di tempo.

Il modello di memoria CUDA, abbiamo visto che esistono delle memorie programmabili che hanno pattern di accesso utilizzabili con le API di sistema e devono essere gestite direttamente, devono essere gestite in maniera diversa per la lettura e la scrittura. Poi ci sono delle memorie non programmabili, le cache L1 e L2.

4.1 Registri

Sono le memorie piu' veloci e piu' piccole. Sono ripartiti tra i warp attivi (assegnati ad ogni thread). Le variabili locali sono quelle che vengono assegnate ai registri senza che sia specificato nessuno dei qualificatori generalmente risiede in un registro. Meno registri usa il kernel, piu' blocchi di thread e' probabile che risiedano sul multiprocessore, quindi meno ne usiamo meglio e'.

4.2 Local Memory

E' una memoria lenta come la global memory. E' una memoria locale ai thread ed e' usata per contenere le variabili automatiche (grandi) non contenute nei registri:

- Array locali i cui indici non possono essere determinati a compile-trasferimento
- Strutture locali grandi che consumano troppo spazio registro
- Ogni variabile che non puo' essere allocata nei registri a causa numero limitato (register spilling)

Risiede nella device memory, pertanto gli accessi hanno stessa latenza e ampiezza di banda della global memory e sono soggetti anche agli stessi vincoli di coalescenza. Il compilatore nvcc si preoccupa della sua allocazione e non e' controllata dal programmatore.

4.3 Constant Memory

Risiede in device memory ed ha una cache dedicata in ogni SM. E' ideale per ospitare dati a cui si accede in modo uniforme e in sola lettura. Una variabile dichiarata come `__constant__` viene dichiarata con scope globale, ad di fuori di qualsiasi kernel. L'utilizzo migliore come dice il nome e' allocarci dati che devono essere distribuiti tra tanti kernel.

```
1 cudaError_t cudaMemcpyToSymbol(const void *symbol, const void *src,
    size_t count);
```

E' pari a 64KB per tutte le compute capability.

4.4 Texture Memory

E' una memoria in sola lettura, dotata di cache e risiede in device memory. E' particolarmente utile perche' include un supporto hw efficiente per filtraggio o interpolazione floating-point nel processo di lettura dei dati. E' ottimizzata per la localita' spaziale 2D, quindi dati espressi sotto forma di matrici. I thread di un warp che usano la texture memory per accedere a dati 2D hanno migliori prestazioni rispetto a quelle standard.

4.5 Shared Memory

E' una memoria programmabile on-chip con un bandwidth molto piu' alto e minor latenza della local o global memory. E' suddivisa in moduli della stessa ampiezza, chiamati bank che possono essere acceduti da un thread alla volta. Abbiamo 32 banchi di memoria che devono essere utilizzati in maniera opportuna per essere acceduti simultaneamente. Ogni SM ha una quantita' limitata di shared memory che viene ripartita tra i blocchi di thread. Bisogna farne un uso parco ancora una volta per non limitare il numero di warp attivi. Con i blocchi di thread condivide anche lifetime e scope. E' fondamentale per la comunicazione inter-thread di un blocco, poiche' permette di scambiare dati in modo rapido e con bassa latenza. L'accesso deve essere sincronizzato per mezzo delle seguente CUDA runtime call:

```
1 __syncthreads();
```

4.5.1 Organizzazione

La SMEM dell'arch Fermi sono suddivisi in blocchi da 4 byte (chiamate word) da 32 bit (0 64). Ogni word puo' contenere tipi base (1 int, 1 float, 2 short, 4 char). la bandwidth e' di 32 bit ogni 2 cicli di clock. Nei 32 banchi andiamo ad accedere simultaneamente a 32 word, ed e' l'accesso piu' veloce che possiamo avere:

4.5.2 SMEM runtime

Viene ripartita tra tutti i blocchi residenti in un SM. Maggiore e' la shared memory richiesta da un kernel, minore e' il numero di blocchi attivi concorrenti. Il contenuto della shared memory ha lo stesso lifetime del blocco cui e' stata assegnata. L'accesso e' per warp: caso migliore 1 transazione x 32 thread, caso peggiore 32 transazioni.

4.5.3 Pattern di accesso

Se un'operazione di load o store eseguita da un warp richiede al piu' un accesso per bank, si puo' effettuare in una sola transizione il trasferimento dati dalla shared memory al warp.

- Caso broadcast: un singolo valore letto da tutti i thread in un warp da un singolo bank
- Caso parallelo: un singolo valore letto da un singolo bank. In questo caso le cose vanno bene, per ogni thread accediamo al corrispondente banco di memoria.
- Conflitto doppio: tutti i thread di un warp richiedono una word di indice doppio il threadId.x
- Nessun conflitto: strutture di 3 word come, ad esempio, le terne di punti nello spazio.

4.5.4 Conflitto di accesso

Il bank conflict e' quando diversi indirizzi di shared memory insistono sullo stesso bank. L'hw effettua tante transizioni quante ne sono necessarie per risolvere il conflitto, riducendo le prestazioni.

4.5.5 Configurazione SMEM / cache L1

Ogni SM ha 64 Kb di memoria on-chip, la shared memory e L1 cache condividono questa risorsa HW. Cuda fornisce due metodi per configurare la L1 cache e la shared memory:

```
1 cudaError_t cudaDeviceSetCacheConfig(cudaFuncCache cacheConfig);
```

Dove l'argomento cacheConfig specifica come deve essere ripartita la memoria:

- cudaFuncCachePreferNone: no preference
- cudaFuncCachePreferShared: prefer shared memory 48KB shared e 16 Kb L1
- cudaFuncCachePreferL1: prefer L1 cache
- cudaFuncCachePreferEqual: equal preference

4.5.6 Osservazioni

Il latency hiding e' sempre una cosa indesiderata, il ritardo che puo' incorrere alla SMEM e all'ottenimento dei dati non e' in generale un problema anche in caso di conflitti di banchi, perche' se si riesce a tenere alto il numero di warp attivi, thread in esecuzione e blocchi letti in esecuzione si ha sempre un sostituto nel momento in cui un warp va in attesa. Inter-block conflict, non esiste un conflitto tra thread appartenenti a blocchi differenti, il problema sussiste solo a livello di warp dello stesso blocco. Il modo piu' semplice per avere prestazioni elevate e' quello di fare in modo che un warp acceda a word consecutive in memoria shared. Caching: con lo scheduling efficace le prestazioni anche in presenza di conflitti a livello di SMEM sono di gran lunga migliori se paragonate a quelle in cui il cammino che i dati percorrono passa attraverso la cache L2 o peggio, arriva in global memory.

4.5.7 Allocazione statica delle SMEM

Una variabile in shared memory puo' anche essere dichiarata sia locale a un kernel sia globale in un file sorgente. E' dichiarata con il qualificatore `__shared__`. Puo' essere dichiarata sia staticamente sia dinamicamente.

4.5.8 Allocazione dinamica della SMEM

Se la dimensione non e' nota a tempo di compilazione e' possibile dichiarare una variabile adimensionale con la keyword `extern`:

- Dinamicamente si possono allocare solo array 1D
- puo' essere sia internal al kernel sia esterna

4.5.9 Uso tipico

- Caricare i dati dalla device memory alla shared memory
- Sincronizzare i thread del blocco al termine della copia che ognuno effettua sulla shared memory (cosi' che ogni thread possa elaborare dati certi nel prosieguo)
- Elabora i dati in shared memory
- Sincronizza per essere certi che la shared memory contenga i risultati aggiornati
- Scrivi i risultati dalla device memory alla host memory

4.5.10 Prodotto matriciale con SMEM

Nella versione senza memoria shared ogni thread e' responsabile del calcolo di un elemento della matrice prodotto. Dobbiamo tenere conto che abbiamo una suddivisione logica dei thread che stanno in un blocco e degli elementi in memoria corrispondenti. Passi con shared memory:

- Occorre caricare in memoria shared i dati relativi ad ogni blocco prima di svolgere le somme e i prodotti
- Ogni thread accumula i risultati di ognuno di questi prodotti (scandendo tutti i blocchi) in un registro
- Alla fine scrive su global memory

Qui l'idea e' di trasferire gli elementi di riga e colonna delle due matrici in shared memory perche' dobbiamo accedere ad ogni elemento tante volte e quindi l'accesso a quella memoria sarebbe piu' veloce.

L'idea e':

- Caricare il primo tile
- Effettuare le somme parziali per ogni cella

- Riptere per il numero di tile contenuti nelle zone delle matrici da caricare
- Scrivere i risultati dalla shared memory alla global memory

4.5.11 Prodotto convolutivo con SMEM

La convoluzione e' un prodotto tra due vettori, tendenzialmente un vettore piu' grande ed uno piu' piccolo chiamato kernel o maschera.

Una volta che abbiamo la scrittura matematica l'istanziamento e' con vettori, nell'esempio vediamo il segnale e la maschera, viene evidenziato il segnale centrale perche' e' quello in output della convoluzione: Occorre fare shiftare la maschera con uno stride = 1 e calcolare prodotti e somme.

Da notare che ci sono casi particolari, nell'esempio partiamo dal terzo elemento perche' c'e' corrispondenza reale con la maschera, si potrebbe anche adottare approcci diversi relativamente a dati mancanti.

Questa cosa si estende naturalmente a piu' dimensioni, questo e' l'esempio del 2D:

Ha senso pensare all'uso della shared memory perche' alcune celle della matrice di input vengono riutilizzate più volte durante il calcolo della convoluzione (tutte quelle sotto la maschera sicuramente).

Questo e' un problema ideale per il parallel computing:

- Determinare la mappa tra thread ed elementi di output
- Semplice approccio nei casi 1D e 2D e' organizzare i thread in grid corrispondenti in modo tale che ogni thread calcoli un elemento della matrice di convoluzione
- Problema: non soddisfa per inefficienza negli accessi alla global memory

Tiling

Divido i dati in blocchi (ad esempio, 16 elementi in 4 blocchi da 4 thread), ogni thread nel blocco fa un prodotto della convoluzione. Per ridurre l'accesso alla global memory, in cache/memoria condivisa si tengono i dati a cui l'accesso è fatto più frequentemente, ovvero i valori del "blocco di dati" assegnato al block (tutti i thread devono calcolare sullo stesso insieme di dati, o quasi), tenendo conto della dimensione della maschera (serve avere i dati "adiacenti" al blocco, quelli che sbordano (sarebbe molto utile un'immagine, si capirebbe subito)).

I dati da caricare in smem sono più dei thread nel blocco ("alone" che va al di fuori del blocco di dati stesso); in smem carico tutti i possibili dati a cui il blocco deve fare l'accesso.

Chi carica che dati in memoria? I dati esterni al blocco potrebbero essere anche più del blocco stesso (maschera "grossa"). La soluzione è dare un ordine ai thread e dividere il più equamente possibile i caricamenti in memoria tra i thread del blocco. Si può fare facendo un tiling dei dati da caricare: il blocco viene "ripetuto" sui dati da caricare, secondo un ordine dei thread.

4.6 Global Memory

Nei computer moderni esiste una gerarchia di memorie per minimizzare latenze e massimizzare throughput. In genere, si ha l'illusione virtuale di una grande memoria, tutta a bassa latenza, anche se la memoria con effettivamente bassa latenza è poca e si ha una memoria ad alta capacità e alta latenza.

All'interno delle GPU abbiamo, dalla latenza più alta alla più bassa:

- Device Memory
- L2 Cache
- L1/shared
- Registers

Le gerarchie di memorie, comprese quelle CUDA, fanno fede ai principi di:

- **Località spaziale:** se l'istruzione all'indirizzo i viene eseguita, probabilmente dopo verrà eseguita quella all'indirizzo $i + \Delta i$
- **Località temporale:** se un'istruzione viene eseguita al tempo t , probabilmente verrà eseguita anche al tempo $t + \Delta t$ (dove Δt è piccolo)

Registers: Le memorie più veloci in assoluto, con lifetime del kernel. Vengono ripartiti tra i warp attivi, le variabili dichiarate nel codice device senza qualificatori generalmente risiedono in un registro.

Meno registri usa il kernel, più blocchi di thread è probabile che risiedano sull'SM (il compilatore usa un'euristica per ottimizzare questo parametro). **Register spilling:** se si usano più registri di quelli consentiti le variabili si riversano nella local memory.

Local Memory: Si tratta di una memoria *lenta* (collocata off-chip, alta latenza, bassa bandwidth). Si tratta di una memoria locale ai thread.

Usata per contenere le variabili automatiche (grandi) non contenute nei registri, o per le variabili al di fuori causa spilling. La local memory risiede nella device memory, pertanto gli accessi hanno stessa latenza e ampiezza di banda della global memory e sono soggetti anche agli stessi vincoli di coalescenza.

Da CC 2.0 ci sono parti poste in cache L1 a livello di SM e in cache L2 a livello di device. Il compilatore `nvcc` si preoccupa della sua allocazione e non è controllata dal programmatore.

Constant Memory: Risiede nella device memory (64K per tutte le CC) ed ha una cache dedicata in ogni SM (8K). Definibile tramite l'attributo `__constant__`.

Ospita dati in sola lettura, ideale per accessi uniformi. Ha scope globale, va dichiarata al di fuori di qualsiasi kernel e viene dichiarata staticamente, quindi è visibile a tutti i kernel nella stessa unità di compilazione.

La constant memory può essere inizializzata dall'host usando:

```
1 cudaError_t cudaMemcpyToSymbol(const void* symbol,  
2   const void* src, size_t count)
```

Lavora bene quando tutti i thread di un warp leggono dallo stesso indirizzo di memoria (raggiunge l'efficienza dei registri); se i thread di un warp leggono da indirizzi diversi allora le letture vengono serializzate, riducendo l'efficienza.

Texture Memory: Risiede nella device memory e (può avere) una read-only cache per-SM ed è acceduta solo attraverso di essa. La cache include un supporto hardware efficiente per filtraggio o interpolazione floating-point nel processo di lettura dei dati.

Ottimizzata per la località spaziale 2D, quindi dati espressi sotto forma di matrici. I thread in un warp che usano la texture memory per accedere a dati 2D hanno migliori prestazioni rispetto a quelle standard, quindi è adatta per applicazioni in cui servono classiche elaborazioni di immagini/video. Per altre applicazioni l'uso della texture memory potrebbe essere più lento della global memory.

Global Memory: La più grande, con più alta latenza e più comunemente usata memoria su GPU. Ha scope e lifetime globale (da qui il nome). Dichiarazione (codice host):

Statica	<code>__device__ int a[N];</code>
Dinamica	<code>cudaMalloc((void **)&d_a, N); cudaFree(d_a);</code>

Corrisponde alla memoria fisica, con "global" si intende una divisione logica. L'accesso da parte di thread appartenenti a blocchi distinti può potenzialmente portare a modifiche incoerenti. La global memory è accessibile attraverso transazioni da 32, 64, o 128 byte; le transazioni avvengono solo per gruppi di valori, non si può accedere a un valore singolo.

I valori contenuti nella memoria allocata non sono inizializzati, ma si possono inizializzare con dati provenienti dall'host (`cudaMemcpy()`) oppure con un valore specifico

```
1 cudaError_t cudaMemcpy(void* devPtr, int value, size_t count)
```

Assegna il valore `value` a tutti gli indirizzi contenuti nel blocco di memoria.

La memoria allocata e' opportunamente allineata per ogni tipo di variabile. La `cudaMalloc()` restituisce `cudaErrorMemoryAllocation` in caso di fallimento.

Lo **specificatore** `__device__` indica una variabile che risiede unicamente sul device. Risiede nella memoria globale (e quindi oggetti distinti per device diversi), ha il lifetime del contesto CUDA in cui è stata creata. Può essere acceduta da tutti i thread e dall'host tramite la libreria runtime:

- `cudaGetSymbolAddress()`, `cudaGetSymbolSize()`: per ottenere indirizzo e dimensione di una variabile, rispettivamente
- `cudaMemcpyToSymbol()`, `cudaMemcpyFromSymbol()`: per copiare verso e da una variabile, rispettivamente

Riassunto dichiarazione di variabili:

QUALIFIER	VARIABLE	MEMORY	SCOPE	LIFESPAN
	<code>float var</code>	Register	Local	Thread
	<code>float var[100]</code>	Local	Local	Thread
<code>__shared__</code>	<code>float var</code>	Shared	Block	Block
<code>__device__</code>	<code>float var</code>	Global	Global	Application
<code>__constant__</code>	<code>float var</code>	Constant	Global	Application

4.6.1 Pinned memory

La pinned memory (o page-locked memory) in CUDA è una tecnica che serve per ottimizzare il trasferimento dei dati tra la memoria del sistema (RAM) e la memoria della GPU (VRAM).

Si vuole evitare il page fault della virtual memory (CPU, di default la memoria host allocata è paginabile). Esistono delle primitive per definire una memoria pinned, ovvero viene tolta la pagina dal meccanismo di virtualizzazione in modo che l'host non possa "toglierla" mentre il device la deve usare. Blocca la memoria in modo da poter fare trasferimenti asincroni al device.

Una volta "pinnata", la memoria non sparirà dal sistema di virtualizzazione automatico della memoria host, quindi si può lavorare su quella memoria in maniera asincrona. La pinned memory può essere acceduta direttamente dal device, in modalità asincrona. Può essere letta e scritta con una bandwidth più alta rispetto alla memoria paginabile.

Da notare che eccessi di allocazione di pinned memory potrebbero far degradare le prestazioni dell'host (ridurre la memoria paginabile inficia l'uso della virtual memory), Per allocare esplicitamente memoria pinned:

```
1 cudaError_t cudaMallocHost(void **devPtr, size_t count);
```

E per deallocarla:

```
1 cudaError_t cudaFreeHost(void *devPtr);
```

Questa allocazione sostituisce la malloc "normale", su host. Rende i trasferimenti host-device significativamente più veloci, al costo di un tempo più alto di allocazione.

4.6.2 Unified Virtual Addressing UVA

Si vuole avere un unico spazio di indirizzamento tra CPU e tutte le GPU. Tutti i puntatori (CPU e GPU) appartengono allo stesso spazio di indirizzi virtuali, di conseguenza è possibile passare un puntatore da host a device e viceversa senza ambiguità, entrambi possono "capire" a cosa punta quell'indirizzo.

La unified memory è una memoria "*comoda*", fornisce un puntatore unico per tutte le CPU e GPU presenti nel sistema. Spazio di indirizzamento unico per CPU e GPU.

Con "**Managed Memory**" si fa riferimento ad allocazioni della unified memory. All'interno di un kernel si possono usare entrambi i tipi di memoria:

- managed memory, controllata dal sistema
- un-managed memory, controllata esplicitamente dall'applicazione

Tutte le operazioni CUDA valide sulla memoria del dispositivo sono valide anche sulla memoria managed.

Per fare allocazione dinamica:

```
1 cudaError_t cudaMallocManaged(void **devPtr, size_t size,  
2 unsigned int flags=0)  
3
```

"rimpiazza" `cudaMalloc`, la `flag` indica chi condivide il puntatore con il device:

- `cudaMemAttachHost`: solo la CPU
- `cudaMemAttachGlobal`: anche tutte le altre GPU

Nuova **keyword**: `__managed__`, si tratta di un qualifier che denota scope globale, accessibile da CPU e GPU.

Con l'uso "misto" di memoria bisogna porre attenzione alla sincronizzazione tra CPU e GPU, onde evitare problemi.

4.6.3 Pattern di Accesso alla Global Memory

Gli accessi alla memoria del dispositivo possono avvenire in transazioni da 32, 64 o 128 byte. Le applicazioni GPU tendono (a volte) ad essere limitate dalla memory bandwidth, quindi massimizzare il throughput effettivo è importante.

In generale, per rendere efficienti le transazioni in memoria:

- minimizzare il numero di transazioni per servire il massimo numero di accessi
- considerare che il numero di transazioni e throughput ottenuto variano con la CC

Per migliorare le prestazioni in lettura e scrittura occorre ricordare che:

- le istruzioni vengono eseguite a livello di warp e gli accessi in memoria dipendono dalle operazioni svolte nel warp
- per un dato indirizzo si esegue un'operazione di loading o storing (gestione diversa)
- i 32 thread presentano una singola richiesta di accesso, che viene servita da una o più transazioni in memoria

In base a come sono distribuiti gli indirizzi di memoria, gli accessi alla stessa possono essere classificati in pattern distinti. Tutti gli accessi a memoria globale passano dalla cache L2, molti passano anche dalla L1. Se entrambe le memorie vengono usate gli accessi sono da 128 byte, altrimenti, se viene usata solo la L2, gli accessi sono a 32 byte.

Per le architetture che usano cache L1, queste possono essere esplicitamente abilitate o disabilitate a compile time.

Bisogna rispettare allineamento e coalescenza per sfruttare al meglio le transazioni di memoria; per avere accessi in memoria efficienti è necessario combinare in un'unica transazione accessi multipli a memoria allineati e coalescenti.

Accesso allineato: quando il primo indirizzo della transazione è un multiplo pari della granularità della cache che viene usata per servire la transazione (32 byte per la cache L2 o 128 byte per la cache L1).

Accesso coalescente: quando tutti i 32 thread in un warp accedono a un blocco contiguo di memoria.

In un SM i dati seguono pipeline attraverso i seguenti tre cache/buffer paths dipendentemente da quale tipo di device memory si accede:

- L1/L2 cache
- Constant cache

- Read-only cache

L1/L2 cache rappresenta il default path. Il fatto che un'operazione di load in global memory passi attraverso la cache L1 dipende da CC e compiler options.

Scritture: Le write vengono servite in modo diverso, non viene usata la cache L1, ma le store sono cachate solo in L2, prima di essere inviate alla device memory in segmenti da 32 byte; vengono trasferiti 1,2 o 4 segmenti alla volta.

Quando forzati a fare accessi (letture/scritture) non coalescenti si può usare la shared memory come "passaggio" per rendere le operazioni effettive in memoria coalescenti.

AoS vs SoA: I dati possono essere divisi in:

- Array of Structures AoS:

```
1 struct Particle { float x, y, z; };
2 Particle* P;
3 float x = P[idx].x; // stride = sizeof(Particle)
```

Questo porta a distanza tra accessi (stride) alta, rompendo la coalescenza

- Structure of Arrays SoA:

```
1 float *Px, *Py, *Pz;
2 float x = Px[idx]; // stride = 1
```

In questo modo lo stride è ridotto, riducendo così il numero di transazioni necessarie

TL;DR: Un warp può effettuare accessi

- coalescenti: i 32 thread leggono dati contigui, massima efficienza
- non coalescenti/strided: dati a distanza > 1, possono servire più transazioni per la stessa quantità di dati, fino a 32 diverse

In generale (per CC superiori a 2) le transazioni coprono 128 byte. All'interno di un singolo segmento da 128 byte, la memoria è organizzata in "banks" (4 da 32 byte solitamente), anche se un thread legge solo 4 byte, dovrà trasferire l'intero bank.

La cache L1 serve load/store anche con granularità a 32 byte.

Matrice trasposta con SMEM

Il problema del parallelizzare l'operazione di trasposizione di una matrice è che in lettura gli accessi sono per riga, quindi coalescenti, mentre le scritture sulla matrice trasposta sono per colonna, quindi non coalescenti.

Un'idea per risolvere il problema può essere:

- Caricare dalla global memory alla smem le celle che il blocco corrente deve trasporre, riga per riga
- Leggere una colonna in smem e scrivere una riga in global memory

Esempio:

```
1 __shared__ float tile[BDIMY][BDIMX];
2 // Coordinate originali
3 int y = blockIdx.y * blockDim.y + threadIdx.y;
4 int x = blockIdx.x * blockDim.x + threadIdx.x;
5
6 // ...
7 // Trasferimento in smem e sincronizzazione
8
9 // Nuovi indici
10 int y = blockIdx.x * blockDim.x + threadIdx.y;
11 int x = blockIdx.y * blockDim.y + threadIdx.x;
12
13 // ...
14 // Scrivere sulla matrice output,
15 //     controlli invertiti tra riga e colonna
```

Chapter 5

Ottimizzazione delle prestazioni

5.1 Stream e concorrenza

Ci sono vari gradi di concorrenza in CUDA:

- CPU/GPU concurrency: poiche' sono dispositivi distinti, la CPU e la GPU possono operare indipendentemente una dall'altra
- Memcpy/kernel processing concurrency: grazie al DMA il trasferimento tra host e device puo' avere luogo mentre gli SMs processano i kernel
- Kernel concurrency: piu' kernel possono essere eseguiti simultaneamente sulla GPU, a patto che ci siano risorse sufficienti disponibili, fino a 32 kernel in parallelo
- Grid-level-concurrency: piu' griglie di thread possono essere eseguite simultaneamente sulla GPU, a patto che ci siano risorse sufficienti disponibili. Uso di stream multipli per operazioni indipendenti
- Multi-GPU concurrency: piu' GPU possono essere utilizzate simultaneamente per eseguire kernel diversi o la stessa operazione su dati diversi. Si distribuisce il processo su diverse GPU che eseguono in parallelo.

Il motto della lezione e' muoversi in maniera asincrona, bisogna utilizzare delle specifiche API che garantiscono l'asincronia dando garanzie:

- Lancio dei kernel
- Memory copies tra due indirizzi allo stesso device
- Memory copies da host a device
- Memory copies fatte da funzioni con il suffisso Async
- Memory set function call con il suffisso Async

L'atomo fondamentale nelle GPU e' il warp e le operazioni dentro sono sincrone. I warp sono suddivisioni di blocchi ed e' possibile sincronizzare a livello di blocco i thread. Dal punto di vista della CPU sappiamo quali sono le operazioni sincrone e il comportamento del kernel (asincrono) e come forzare la sincronizzazione dell'host.

5.1.1 CUDA Stream

Uno stream CUDA e' riferito a sequenze di operazioni CUDA asincrone che vengono eseguite sul device nell'ordine stabilito dal codice host. Lo stream gia' esiste senza che lo sapessimo, se non definiamo niente esiste uno stream di default. Le operazioni tipiche includono il trasferimento dati host-device, lancio di kernel, gestioni di eventi verranno demandate agli stream con il grande vantaggio che tutto diventa asincrono e indipendente tra di loro, come se avessimo dei thread diversi sull'host ed ognuno fa la sua cosa. Questo introduce ad un parallelismo a livello di griglia.

Tipi di stream Gli stream CUDA possono essere classificati in tre categorie principali:

- **Stream di default:** se non viene creato uno stream specifico, tutte le operazioni CUDA vengono eseguite nello stream di default. Questo stream è implicito e non richiede alcuna gestione da parte del programmatore.
- **Stream dichiarati esplicitamente**

In generale e' meglio usare stream non nulli quando:

- Si vuole introdurre concorrenza per sovrapporre operazioni articolate tra host e device (si possono spezzare le operazioni)
- Sovrapporre computazioni host e trasferimento dati host-device
- Sovrapporre trasferimento dati host-device e computazioni device
- Computazioni concorrenti su device

Default Stream

Quando vengono lanciate delle API CUDA senza specificare uno stream, queste operazioni vengono automaticamente assegnate allo stream di default. Questo significa che tutte le operazioni eseguite nello stream di default sono serializzate, ovvero vengono eseguite una dopo l'altra, senza sovrapposizioni. Sebbene questo possa semplificare la programmazione, può anche portare a un utilizzo inefficiente delle risorse della GPU, poiché non si sfruttano appieno le capacità di parallelismo offerte da CUDA. E' tutto bloccante sia per host che per device.

```
1  cudaMemcpy(d_out, d_in, size, cudaMemcpyDeviceToDevice);
2  //bloccato
3  kernel_1<<<grid, block, 0, stream>>>(d_out);
4  //bloccato
5  function_A()
6  //bloccante
7  cudaMemcpy(d_out, d_in, size, cudaMemcpyDeviceToDevice);
```

La GPU vede sequenze di operazioni che gli vengono mandate dall'host e non fa altro che accodare queste operazioni ed eseguirle, mentre la CPU puo' continuare a lavorare su altre funzioni.

Stream Espliciti

Quando si desidera avere un controllo più fine sull'ordine di esecuzione delle operazioni CUDA, è possibile creare e utilizzare stream espliciti. Gli stream espliciti consentono di eseguire operazioni in parallelo e di sovrapporre trasferimenti di dati e computazioni. Per creare uno stream esplicito, si utilizza la funzione `cudaStreamCreate()` e si specifica lo stream desiderato durante il lancio delle operazioni CUDA:

```
1  cudaError_t cudaStreamCreate(cudaStream_t *pStream);
2  kernel_name<<<grid, block, sharedMemSize, pStream>>>(argument list)
3  cudaError_t cudaStreamDestroy(cudaStream_t pStream);
```

Passiamo un handle al kernel con cui gestisce lo stream.

Siccome stiamo parlando di operazioni asincrone guardiamolo anche dal punto di vista della memoria, diciamo che vogliamo l'impiego della memoria asincrona, vogliamo la memoria pinned:

```
1 cudaError_t cudaMallocHost(void **ptr, size_t size);  
2 cudaError_t cudaHostAlloc(void **ptr, size_t size, unsigned int  
    flags);
```

Alloca su host memoria non paginabile, se $\text{flag} = 0$ il comportamento delle due API e' uguale.

Questo fa la copia asincrona dei dati tra host e device:

```
1 cudaError_t cudaMemcpyAsync(void *dst, const void *src, size_t  
    count, cudaMemcpyKind kind, cudaStream_t stream);
```

Si ha ancora bisogno di sincronizzare le operazioni in uno stream. Il blocco dell'host su un determinato stream si fa:

```
1 cudaError_t cudaStreamSynchronize(cudaStream_t stream);
```

Forza il blocco dell'host fino a che tutte le operazioni nello stream sono state completate.

Lo stream puo' essere anche monitorato rispetto al suo livello di esecuzione, possiamo controllare se le operazioni sono completate ma non forziamo il blocco dell'host in caso negativo, ritorna `cudaSuccess` se le operazioni sono completate e `cudaErrorNotReady` altrimenti (mettiamo nella coda FIFO la `cudaStreamQuery`):

```
1 cudaError_t cudaStreamQuery(cudaStream_t stream);
```

Sovrapporre kernel e trasferimento dati

Il device e' capace di avere copie ed esecuzioni concorrenti, questo puo' essere indagato con il campo `deviceOverlap` della struct `cudaDeviceProp`. Per fare questo e' necessario che il kernel e il trasferimento appartengano a differenti non-default stream e dobbiamo usare la pinned memory. Un uso tipico e' se abbiamo un array di N elementi, possiamo spezzarlo in gruppi da M elementi, se il kernel opera indipendentemente su tutti gli elementi ogni gruppo puo' essere elaborato a parte, il numero di $nStreams = \frac{N}{M}$.

Default Stream prima di CUDA 7

Hanno introdotto uno stream di default per ogni processo host dopo la 7, prima invece c'era un solo stream di default per ogni processo e questo fa una sincronizzazione implicita tra tutte le operazioni. Questo addirittura rende sincrone operazioni anche di thread host diversi.

Sincronizzazione rispetto a NULL-stream

Il NULL-stream e' sempre bloccante, anche all'interno dello stesso host (nella nostra applicazione) e' particolarmente bloccante, e' in mutex con gli altri stream. A meno che non si dica che ogni stream non null prodotto sia non bloccante, e' quindi possibile specificare il comportamento di uno stream non di default rispetto a quello di default

rispetto al blocco. Dal punto di vista dell'host ogni kernel e' asincrono e non bloccante, ma in questo caso kernel2 non parte finche' kernel1 e' completato e cosi' fa 3 con 2, questo perche' si ha il comportamento di default con il NULL-stream:

```
1 kernel_1<<<1, 1, 0, stream_1>>>();
2 kernel_2<<<1, 1>>>();
3 kernel_3<<<1, 1, 0, stream_2>>>();
```

Concorrenza tra stream

Non c'e' nessuna garanzia che due kernel vadano in simultanea, non siamo mai certi di cosa sta accadendo in reale concorrenza sulla GPU, sappiamo pero' per certo che due kernel sullo stesso stream vengono eseguiti in sequenza. La stessa cosa se abbiamo stream bloccanti. Cosa diversa succede se abbiamo `cudaStreamNonBlocking`.

Priorita' negli stream

Una grid con piu' alta priorita' puo' soppiantare un'altra con meno priorita'. Si possono creare stream con priorita' (da CC 3.5). Una grid con più alta priorita' può prelazionare il lavoro già in esecuzione con più bassa priorita'.

Hanno effetto solo su kernel e non su data transfer. Priorita' al di fuori del range vengono riportate automaticamente nel range.

Per creare e gestire uno stream con priorita' si usano le funzioni:

```
1 cudaError_t cudaStreamCreateWithPriority(
2     cudaStream_t* pStream, unsigned int flags, int priority);
```

Crea un nuovo stream con priorita' intera e ritorna l'handle in `pStream`.

```
1 cudaError_t cudaDeviceGetStreamPriorityRange(
2     int *leastPriority, int *greatestPriority);
```

Restituisce la minima e massima priorita' del device (la più alta è la minima).

Host Functions (Callback): Si ha la possibilità di inserire una funzione host, senza introdurre sincronizzazione o interrompere il flusso dello stream. Questa funzione viene eseguita sull'host una volta che tutti i comandi forniti allo stream prima della chiamata sono stati eseguiti.

Chiamare un kernel su più Stream

Il vantaggio dell'usare gli stream risiede nella possibilità di dividere le (stesse) operazioni in "batch", aumentando il parallelismo con la possibilità di avere trasferimenti di memoria (sia H2D che D2H) concorrenti all'esecuzione del kernel stesso.

Vogliamo dividere i dati in blocchi e rendere i trasferimenti paralleli all'esecuzione, aumentando l'occupancy. Sono possibili due schemi di versioni asincrone:

Schema 1:

- loop che chiama, per ogni stream: copia H2D, kernel, copia D2H

Le due versioni sono funzionalmente la stessa cosa. A livello di kernel, l'unica cosa che cambia (come per tutte le volte in cui si suddivide) è da tenere conto dell'offset per considerare la "zona" assegnata a quel kernel in quello stream.

Schema 2:

- loop per copie H2D
- loop per kernel
- loop per copie D2H

5.1.2 CUDA event

Un evento è un marker all'interno di uno stream associato ad un punto del flusso delle operazioni. Serve per controllare se l'esecuzione di uno stream ha raggiunto un dato punto o anche per la sincronizzazione inter-stream. Può essere usato per due scopi base:

- Sincronizzare l'esecuzione di stream
- Monitorare il progresso del device

Le API CUDA forniscono funzioni che consentono di inserire eventi in qualsiasi punto dello stream. Oppure effettuare delle query per sapere se lo stream è stato completato. Quindi questo evento ha un duplice stato, è accaduto oppure no.

Per creare un evento bisogna usare una primitiva di create:

```
1  cudaEvent_t event;  
2  cudaError_t cudaEventCreate(cudaEvent_t *event);  
3  cudaError_t cudaEventDestroy(cudaEvent_t event);
```

Gli eventi nello stream zero vengono completati dopo che tutti i precedenti comandi in tutti gli altri stream sono stati completati. Gli eventi hanno uno stato booleano che indica se l'evento è stato completato o meno.

Sincronizzazione via CUDA event

Una volta creato l'evento si può associare ad uno stream, se non specifico niente viene associato allo stream zero.

```
1  cudaError_t cudaEventRecord(cudaEvent_t event, cudaStream_t stream);
```

Si può usare l'evento per sincronizzare con l'host fino a che l'evento non si verifica:

```
1  cudaError_t cudaEventSynchronize(cudaEvent_t event);
```

L'attività non bloccante è quella di fare una query su un evento:

```
1  cudaError_t cudaEventQuery(cudaEvent_t event);
```

Si può sincronizzare anche tra stream ed eventi, considerando bloccante l'attesa su uno stream:

```
1  cudaError_t cudaStreamWaitEvent(cudaStream_t stream, cudaEvent_t  
    event, unsigned int flags);
```

Sincronizzazione Esplicita: CUDA runtime supporta diversi modi di sincronizzazione esplicita a livello di grid in un programma CUDA, si può sincronizzare rispetto

- al device
- a uno stream
- a un evento all'interno di uno stream
- a diversi stream (tra loro), usando un evento

Si può bloccare l'host fino a che il device non ha completato i task precedenti:

```
1 cudaError_t cudaDeviceSynchronize();
```

Si può bloccare l'host fino a che tutte le operazioni in uno stream sono completate (`cudaStreamSynchronize()`) oppure eseguire un test non-bloccante (`cudaStreamQuery()`):

```
1 cudaError_t cudaStreamSynchronize(cudaStream_t stream);
2 cudaError_t cudaStreamQuery(cudaStream_t stream);
```

Un CUDA event può anche essere usato per sincronizzare host e device:

```
1 cudaError_t cudaEventSynchronize(cudaEvent_t event);
2 cudaError_t cudaEventQuery(cudaEvent_t event);
```

Eventi per misurare il tempo

Esiste una primitiva per misurare il tempo intercorso tra due eventi:

```
1 cudaError_t cudaEventElapsedTime(float *ms, cudaEvent_t start,
    cudaEvent_t end);
```

5.2 Librerie CUDA

Tutte le computazioni implementate nelle librerie CUDA sono accelerate dalla GPU. Per molte applicazioni offrono un buon bilanciamento tra usabilità e prestazioni. Il vantaggio è il minimo sforzo per il porting delle applicazioni su GPU, senza dover scrivere codice CUDA. Le librerie sono ottimizzate per le architetture NVIDIA e sono progettate per essere facili da usare. Inoltre noi non abbiamo nessun costo di mantenimento.

Esempi di librerie CUDA:

Libreria	Dominio
cuFFT (NVIDIA)	Fast Fourier Transforms Linear
cuBLAS (NVIDIA)	Linear Algebra (BLAS Library)
cuSPARSE (NVIDIA)	Sparse Linear Algebra
cuRAND (NVIDIA)	Random Number Generation
NPP (NVIDIA)	Image and Signal Processing
CUSP (NVIDIA)	Sparse Linear Algebra and Graph Computations
CUDA Math Library (NVIDIA)	Mathematics
Trust (terze parti)	Parallel Algorithms and Data Structures
MAGMA (terze parti)	Next generation Linear Algebra

5.2.1 Workflow tipico

- Creare un handle specifico della libreria, un riferimento che consente di accedere alle funzioni della libreria.
- Allocare la device memory per gli input e output alle funzioni della libreria, se necessario, convertire gli input al formato specifico di uso della libreria (numeri complessi)
- Popolare con i dati nel formato specifico
- Configurare le computazioni per l'esecuzione
- Eseguire la chiamata della funzione di libreria che avvia la computazione sulla GPU
- Recuperare i risultati della device memory
- Se necessario, convertire i dati nel formato specifico o nativo dell'applicazione
- Rilasciare le risorse CUDA allocate per la data libreria

5.2.2 Libreria cuBLAS

Sono disponibili diverse funzioni per l'algebra lineare, tra cui:

- Operazioni tra vettori
- Operazioni tra matrici e vettori
- Operazioni tra matrici

Questi sono le basi per lo sviluppo delle LAPACK, che sono librerie di algebra lineare un po' più complesse, che implementano algoritmi per la risoluzione di sistemi lineari, decomposizioni e altro. Tutte le interfacce messe a disposizione sono asincrone sulla GPU.

Usa **column-major order** (leggo le colonne dall'alto verso il basso) perché chiunque ha scritto la libreria è stronzo (colpa di Fortran). Esempio:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \rightarrow [1 \ 4 \ 7 \ 2 \ 5 \ 8 \ 3 \ 6 \ 9] \quad I(r, c) = c \cdot M + r$$

Dove (r, c) sono le coordinate del valore cercato e M è l'altezza della matrice (dimensioni $M \times N$).

Operare con cuBLAS

- Creare un handle `cublasCreateHandle`
- Allocare la memoria per gli input e output usando `cudaMalloc`
- Popolare i dati di input nella memoria device usando `cublasSetVector` e `cublasSetMatrix`

- Eseguire le operazioni desiderate usando le funzioni cuBLAS, ad esempio `cublasSgemv`
- Recuperare i risultati dalla memoria device usando `cublasGetVector` e `cublasGetMatrix`
- Rilasciare le risorse allocate usando `cublasDestroy` e `cudaFree`

Funzioni all'interno di cuBLAS

Le prime funzioni che vediamo sono quelle per trasferire righe e colonne da una matrice A nella memoria del device ad una matrice B nella GPU memory space:

```
1 cublasSetMatrix(int rows, int cols, const float *A, int lda, float
    *B, int ldb)
```

I parametri sono:

- **rows**: numero di righe della matrice A
- **cols**: numero di colonne della matrice A
- **A**: puntatore alla matrice A nella memoria del host
- **lda**: leading dimension della matrice A
- **B**: puntatore alla matrice B nella memoria del device
- **ldb**: leading dimension della matrice B

Le funzioni inverse per trasferire i dati dalla memoria del device alla memoria del host sono:

```
1 cublasGetMatrixAsynch(int rows, int cols, const float *A, int lda,
    float *B, int ldb, cudaStream_t stream)
```

Danno la possibilit  di lavorare in asincrono specificando uno stream.

Se volessi fare la copia di un vettore, potrei usare le seguenti funzioni:

```
1 cublasSetVector(int n, const float *A, int incx, float *B, int incy)
2 cublasGetVector(int n, const float *A, int incx, float *B, int incy)
```

Per gestire la libreria serve un **handle**, il quale si pu  generare tramite

```
1 cublasCreate(cublasHandle_t* handle)
```

Viene passato ad ogni chiamata di funzione della libreria successiva. Al termine

```
1 cublasDestroy(cublasHandle_t* handle)
```

per distruggerlo. Il tipo dell'handle   `cublasHandle_t`. Esiste un tipo `cublasStatus_t` usato per il report degli errori.

Per trasferimenti device-device: copia n elementi da x a y:

```
1 cublasScopy(handle, n, x, incx, y, incy)
```

In generale la libreria segue una naming convention `cublas<T>operation`, dove `<T>` può essere:

- S per parametri di tipo float
- D per double
- C per complex floats
- Z per complex double

Ad esempio, per l'operazione `axpy` le funzioni disponibili sono `cublasSaxpy`, `cublasDaxpy`, `cublasCaxpy`, `cublasZaxpy`.

Si usa un valore di tipo `cublasOperation_t` per indicare operazioni su matrici all'interno di funzioni:

- CUBLAS_OP_N per non-transpose
- CUBLAS_OP_T per transpose
- CUBLAS_OP_C per conjugate transpose

Per fare

$$result = \sum_{i=1}^n x[k] \cdot y[j], \quad k = 1 + (i - 1) \cdot incx, \quad j = 1 + (i - 1) \cdot incy$$

tra vettori `x` e `y` di `n` elementi (dimensione dei tali nella naming convention) e mettere il risultato in `result`

```
1 cublasStatus_t cublasSdot(cublasHandle_t handle, int n,
2   const float *x, int incx, const float *y,
3   int incy, float result)
```

Per fare

$$y[i] = \alpha \cdot x[i] + y[i] \quad \forall i \in n$$

con vettori `x` e `y` di dimensione `n`, risultato nel secondo vettore `y`

```
1 cublasStatus_t cublasSaxpy(cublasHandle_t handle, int n,
2   const float *alpha, const float *x, int incx,
3   const float *y, int incy)
```

Per fare

$$y = \alpha Ax + \beta y$$

dove α e β sono scalari, A è una matrice, x e y sono vettori

```
1 cublasStatus_t cublasSgemv(cublasHandle_t handle,
2   cublasOperation_t trans, int m, int n,
3   const float *alpha, const float *A,
4   int lda, const float *x, int incx,
5   const float *beta, float *y, int incy)
```

Per fare

$$C = \alpha AB + \beta C$$

dove α e β scalari, A , B e C matrici

```
1 cublasStatus_t cublasSgemm(cublasHandle_t handle,  
2   cublasOperation_t transa, cublasOperation_t transb,  
3   int m, int n, int k, const float *alpha,  
4   const float *A, int lda,  
5   const float *B, int ldb,  
6   const float *beta, float *C, int ldc)
```

5.2.3 cuRAND

Vuol dire generare numeri a caso, la storia dei generatori pseudo-casuale di numeri e' infinita. Permette sequenze:

- Pseudo-random: soddisfa proprietà statistiche di una vera sequenza random, ma generata da un algoritmo deterministico
- Quasi-random: sequenza di punti n -dimensionali uniformemente generati secondo un algoritmo deterministico

Si compone di due parti:

- curand.h per l'host
- curand_kernel.h per il device

Host API: Dalla documentazione, passaggi:

1. Crea un **nuovo generatore** del tipo desiderato con `curandCreateGenerator()`
2. Setta i **parametri** del generatore; ad esempio: per settare il seed `curandSetPseudoRandomGeneratorSeed()`
3. Alloca la memoria device con `cudaMalloc()`
4. Genera i valori casuali necessari con `curandGenerate()` (o altre funzioni)
5. Usa i valori
6. Quando non serve più il generatore va distrutto con `curandDestroyGenerator()`

Alcune funzioni per l'host:

- Per creare il generatore

```
1 curandCreateGenerator(&g, GEN_TYPE)
```

Dove il parametro `GEN_TYPE` può essere `CURAND_RNG_PSEUDO_DEFAULT`, oppure `CURAND_RNG_PSEUDO_DEFAULT` (differenze trascurabili)

- Per impostare il seed

```
1 curandSetRandomGeneratorSeed(g, SEED)
```

ma importa poco, uno qualunque va bene (e.g. `time(NULL)`)

- Per generare una distribuzione

```
1 curandGenerate_____ (...)
```

dipende dalla distribuzione che si vuole generare, ad esempio: `curandGenerateUniform(g, src, n)` oppure `curandGenerateNormal(g, src, n, mean, stddev)`.

- Per distruggere il generatore

```
1 curandDestroyGenerator(g)
```

La funzione `curandGenerate()` permette di generare valori in maniera asincrona, molto più veloce per quantità elevate di valori. Usare questa libreria richiederebbe poi di dover passare i dati generati alla GPU (`src` all'interno della funzione è un puntatore host), introducendo overhead. Per risolvere si può usare la Device API.

Device API: Per generare valori sul device:

1. Pre-allocare un set di cuRAND state objects nella device memory per ogni thread (gestiscono lo stato)
2. Opzionale, pre-allocare device memory per tenere i valori generati (se devono poi essere passati all'host o essere mantenuti per kernel successivi)
3. Inizializzare lo stato di tutti gli state objects con una kernel call
4. Chiamare una funzione cuRAND per generare valori casuali usando gli state objects allocati
5. Opzionale, trasferire i valori all'host (se è stata allocata la memoria in precedenza)

Il vantaggio di generare valori sul device è che non dobbiamo trasferire i numeri al device, ma li generiamo in loco e i thread li consumano lì'.

5.2.4 cuFFT

La libreria cuFFT (CUDA Fast Fourier Transform) permette di eseguire trasformate di Fourier in modo efficiente sulla GPU. Supporta sia segnali unidimensionali che multidimensionali e offre diverse opzioni per la pianificazione e l'esecuzione delle trasformate.

La libreria prevede che venga definito un piano `cufftPlan1d()` per le trasformate unidimensionali, `cufftPlan2d()` per le trasformate bidimensionali e `cufftPlan3d()` per le trasformate tridimensionali. Questi piani contengono informazioni sulla dimensione del segnale e sul tipo di trasformata da eseguire.

```
1 cufftResult cufftPlan1d(cufftHandle *plan, int nx, cufftType type,
    int batch);
```

C'è la versione batched per lavorare su più batch distinti e lavorare in parallelo facendo sovrapposizioni.

Workflow

- creare e configurare il piano cuFFT plan
- Allocare la memoria con `cudaMalloc`
- Popolare la memoria con i campioni del segnale di input con `cudaMemcpy`
- Eseguire il piano usando la funzione `cufftExec*`
- Recuperare i risultati dalla memoria device con `cudaMemcpy`
- Rilasciare le risorse allocate con `cufftDestroy` e `cudaFree`

Chapter 6

Numba

6.1 Numba

L'idea è quella di avere codice python che sfrutta runtime CUDA per sfruttare il parallelismo esposto dalla GPU.

pyCUDA: L'idea di PyCUDA è di fare un "wrap" del codice C.

PyTorch: Uno dei più usati, spesso per il ML, orientato anche a non programmatori CUDA (generalmente chi lo usa lo fa in modo trasparente, senza sapere il funzionamento della GPU sottostante). L'idea è quella di replicare librerie all'interno di torch, usando CUDA (e.g., NumPy).

Rapids: Azienda che produce molteplici software, include alcune librerie come CuPy (NumPy e SciPy su GPU).

6.1.1 Numba for CPU

Per la CPU, Numba nasce con l'idea di accelerare tramite parallelismo su CPU. Numba è un package compilato Just-In-Time, non richiede uno step di compilazione dedicato, compila solo le funzioni che servono, usa LLVM per la traduzione a linguaggio macchina. Numba si integra con l'ecosistema python (NumPy, Pandas, ...) e permette di usare solo codice Python per fare *tutto*.

Decoratori in Python: Un decoratore è un oggetto usato per modificare una funzione, metodo o classe per trasformarla. Per Numba, si usa il decoratore `@numba.jit` prima della funzione.

Ufuncs: Le funzioni universali sono un concetto introdotto da NumPy per indicare funzioni che operano elemento per elemento su un array NumPy. Permettono di eliminare la necessità di scrivere esplicitamente i cicli `for` e sono (solitamente) compilate in C per efficienza.

Vectorize Decorator: Le operazioni vettorizzate eliminano i loop espliciti e permettono:

- maggiore velocità: non bisogna interpretare più volte la stessa operazione e le computazioni possono avvenire in parallelo
- minore overhead di memoria: minimizzando le variabili temporanea ed effettuando gli scambi in place, con conseguente miglior utilizzo della cache (e quindi performance)
- miglior uso del modello SIMD della CPU
- si può anche avere accelerazione GPU

Per usare il decoratore:


```

1 @vectorize([int64(int64,int64), float32(float32,float32)])
2 def numba_dtype_sum(x, y):
3     return x + y

```

Chiamare una funzione @jit:

- determinare il tipo degli argomenti forniti
- controllare se esiste una versione compilata a codice macchina e, nel caso, utilizzarla
- compilare, se necessario, una versione in linguaggio macchina ottimizzata
- convertire i parametri, questi vengono convertiti in valori compatibili con il codice macchina
- eseguire il codice ottimizzato, viene chiamata direttamente la funzione compilata
- convertire il risultato in un valore compatibile con Python

6.1.2 Numba for GPU

Anche qui si possono costruire funzioni universali e usare vettorizzazione, per poi dire che il target è CUDA, "dirottando" la compilazione verso CUDA tramite un semplice decoratore. Si tratta però di un modello piuttosto rigido.

Ma si possono anche definire kernel sulla GPU con `@cuda.jit`. Si possono lanciare kernel con `kernel[nBlocks, nThreads](args)`, supportano shared, pinned e local memory. Viene usato `nvcc` per compilare.

Dichiarazione dei kernel: Non possono tornare esplicitamente valori, quindi tutti i dati devono essere scritti su array passati alla funzione. Va dichiarata esplicitamente la thread hierarchy (numero di grid e blocchi). Una funzione può essere chiamata più volte, con diversi parametri e thread hierarchy, ma viene compilata una sola volta.

Thread Hierarchy: I valori della gerarchia possono essere ottenuti con:

- `numba.cuda.threadIdx`: indice del thread all'interno del blocco, da 0 a `numba.cuda.blockDim-1`
- `numba.cuda.blockDim`: dimensione del blocco di thread, per come dichiarato per l'istanza del kernel
- `numba.cuda.blockIdx`: indice del blocco all'interno della griglia di thread, da 0 a `numba.cuda.gridDim-1`
- `numba.cuda.gridDim`: dimensione della griglia di blocchi, numero totale di blocchi lanciati per l'istanza del kernel

Mentre le posizioni assolute si possono ottenere con:

- `numba.cuda.grid(ndim)`: torna la posizione assoluta del thread all'interno dell'intera griglia di blocchi; `ndim` deve corrispondere al numero di dimensioni dichiarate per l'istanza del kernel, se `i=1` torna un solo intero, altrimenti una tupla contenente quel numero di interi
- `numba.cuda.gridsize(ndim)`: torna la dimensione assoluta ("forma") in thread dell'intera griglia di blocchi

Vale la pena ricordare che in Python il tipo di default è `f64` (o qualcosa di simile, idk), quindi senza specificare il tipo anche la libreria userà quello. Se tale precisione non è richiesta (in genere in CUDA non si lavora con `f64`) bisogna specificare esplicitamente il tipo. Ad esempio usando `.astype(np.float32)`.

Funzioni device: Si possono scrivere funzioni richiamabili solo dall'interno del device, con il decoratore `@cuda.jit(device=True)`, ad esempio:

```
4 @cuda.jit(device=True)
5 def a_device_function(a, b):
6     return a + b
```

6.1.3 Gestione della memoria

Numba trasferisce automaticamente gli array NumPy quando viene invocato il kernel, ma lo può fare solo in modo "conservativo", quindi trasferisce sempre la memoria device *back to the host* quando finisce. Si possono gestire manualmente i trasferimenti per evitare di passare inutilmente array read-only.

Api per allocare e trasferire:

- Alloca un ndarray device (similmente a `numpy.empty()`)

```
7 numba.cuda.device_array(shape, dtype=...,
8     strides=..., stream=0)
```

- Chiama `device_array()` con informazioni dall'array

```
9 numba.cuda.device_array_like(ary, stream=0)
```

- Alloca e trasferisce un ndarray numpy o uno scalare strutturato al device

```
10 numba.cuda.to_device(obj, stream=0, copy=True, to=None)
```

Pinned e mapped memory: La memoria pinned a mapped si può gestire tramite:

- Un context manager per pinnare temporaneamente una sequenza di ndarray host

```
11 numba.cuda.pinned(*arylist)
```

- Alloca un ndarray con un buffer pinnato (pagelocked)

```

12 numba.cuda.pinned_array(shape, dtype=...,
13     strides=..., order='C')

```

- Chiama un array pinned con le informazioni dall'array

```

14 numba.cuda.pinned_array_like(ary)

```

- Un context manager per mappare temporaneamente una sequenza di ndarray host

```

15 numba.cuda.mapped(*arylist, **kws)

```

Deallocazione: In generale, la deallocazione è gestita in modo automatico, tracciata per-contex. Nei casi di gestione asincrona la deallocazione automatica potrebbe causare problemi, quindi `numba.cuda.defer_cleanup()` permette di fermare la deallocazione (usata tramite blocco `with`).

Esempio:

```

16 with defer_cleanup():
17     # all cleanup is deferred in here
18     do_speed_critical_code()
19 # cleanup can occur here

```

Static shared memory:

- Alloca un array shared della dimensione e tipo specificato; la funzione deve essere chiamata dall'interno del device

```

20 numba.cuda.shared.array(shape, type)

```

`shape` può essere un intero o una tupla di interi, rappresenta le dimensioni dell'array; deve essere una espressione semplice

- Sincronizza tutti i thread all'interno dello stesso blocco

```

21 numba.cuda.syncthreads()

```

Dynamic shared memory: Per usare la memoria shared dinamica, nel kernel va dichiarato un array shared di dimensione 0; esempio:

```

22 @cuda.jit
23 def kernel_func(x):
24     dyn_arr = cuda.shared.array(0, dtype=np.float32)

```

Durante la chiamata a kernel va specificata la dimensione in byte della shared memory:

```

25 kernel_func[32, 32, 0, 128](x)

```

L'ultimo parametro è la shared memory.

Tutta la memoria dinamica diventa un alias allo stesso array; dichiarando più array dinamici quello che succede sarà che ci saranno solamente più puntatori ad uno stesso array, con interpretazioni differenti (stessi dati).

Una soluzione al problema può essere invertire un array, raddoppiando la dimensione totale durante la chiamata:

```
26 f32_arr = cuda.shared.array(0, dtype=np.float32)
27 i32_arr = cuda.shared.array(0, dtype=np.int32)[1:]
```

In questo modo uno viene letto dall'inizio, uno dal fondo. Servono visioni disgiunte degli array.

6.1.4 Atomic Operations

Si possono usare operazioni atomiche per evitare race conditions, le quali possono accadere nei casi di **read-after-write** (un thread prova a leggere una cella di memoria nello stesso momento in cui un altro sta scrivendo) oppure **write-after-write** (due thread provano a scrivere nello stesso momento).

Il namespace per le operazioni atomiche è la classe `numba.cuda.atomic`. Ad esempio, `add(ary, idx, val)` svolge l'operazione atomica `ary[idx] += val`; supportata su i32, f32 e f64.

6.1.5 Streams

Gli **stream** possono essere passati alle funzioni, vengono usati durante la configurazione per il lancio del kernel, in modo che le operazioni siano eseguite in maniera asincrona.

La funzione

```
28 numba.cuda.stream()
```

crea uno stream CUDA, il quale rappresenta la coda dei comandi per il dispositivo.

La funzione

```
29 numba.cuda.default_stream()
```

restituisce lo stream di default. Solitamente, il default stream si comporta in maniera legacy o per-thread on base alla API CUDA in uso.

La funzione

```
30 Stream.synchronize()
```

permette la sincronizzazione all'interno dello stream, i.e., aspetta che tutti i comandi all'interno dello stream vengano eseguiti.

Esempio di uso degli stream:

```
31 # Crea gli stream
32 stream1 = cuda.stream()
33 stream2 = cuda.stream()
34 # Partizione e copia dei dati, per ogni stream
35 # ...
36 # Lancio del kernel, per ogni stream
37 kernel[blocks_per_grid, threads_per_block, stream1]()
```

```
38 kernel[blocks_per_grid, threads_per_block, stream2]()
39 # Porta i dati sull'host
40 # ...
41 # Aspetta il termine degli stream
42 stream1.synchronize()
43 stream2.synchronize()
```

Chapter 7

PyTorch

PyTorch è un framework open source per il ML, sviluppato principalmente da Facebook AI, combina un backend accelerato dalla GPU con frontend Python.

Il core di PyTorch viene usato per implementare tensori, strutture dati, operatori CPU e GPU, primitive parallele e calcoli differenziali. Il core è la parte più computazionalmente intensiva, ma può essere implementata in C++ per le performance.

Si ha una separazione netta tra control e data flow: il controllo è solamente Python, il data flow è codice C++ il quale può essere eseguito sia su CPU che GPU.

PyTorch è compatibile con librerie e pacchetti popolari come NumPy, SciPy e Numba.

7.1 Tensori

PyTorch funziona principalmente attraverso i **tensori**, strutture dati simili ad array e matrici, ma n dimensionali. I tensori vengono usati per codificare input, output e parametri del modello.

In breve, sono "matrici", senza un limite stretto sul numero di dimensioni.

I tensori possono essere inizializzati in diversi modi:

- Per creare un **tensore vuoto**:

```
44 t = torch.empty(x, y)
```

i parametri sono le dimensioni

- Ma spesso li si vuole inizializzare con un **valore**, tipicamente 0, 1 oppure casuali:

```
45 t = torch.zeros(x, y)
46 t = torch.ones(x, y)
47 t = torch.rand(x, y)
```

- Per inizializzare un tensore con la **stessa forma di un altro**:

```
48 t = torch.empty_like(another_tensor)
49 t = torch.zeros_like(another_tensor)
50 t = torch.ones_like(another_tensor)
51 t = torch.rand_like(another_tensor)
```

- Per creare un tensore inizializzato tramite una **struttura dati esistente**:

```
52 vector = torch.tensor([7, 7])
53 matrix = torch.tensor([[7, 8], [9, 10]])
```

- Si può creare un **vettore 1D** con tutti i valori da 0 a `end-1` tramite `torch.arange(end)`, lo si può poi **vedere** o **ridimensionare** con altre forme:

```
54 t = torch.arange(end).view(x, y)
55 t = torch.arange(end).reshape(x, y)
```

Il tipo di default è `f32`, ma lo si può sovrascrivere tramite il parametro `dtype`. Ad esempio:

```
56 t = torch.ones((x, y), dtype=torch.int16)
```

All'interno della memoria fisica, i tensori sono sempre memorizzati in maniera linearizzata e contigua. Lo si può anche vedere tramite

```
57 t.storage()
```

7.2 Operazioni

All'interno della libreria sono presenti numerose (> 100) operazioni di aritmetica, algebra lineare, operazioni su matrici, ...

Permette indicizzamento e slicing in maniera analoga a NumPy. Si può anche indicizzare in più dimensioni tramite il simbolo ":"

```
58 t[:, 0, 0]
```

restituisce l'elemento in alto a sinistra di ogni elemento presente all'interno del tensore. Ovviamente si possono anche accedere elementi arbitrari in ogni dimensione.

Broadcasting: Da NumPy viene copiato il broadcasting, se le dimensioni non coincidono, il risultato "estende" la dimensione minore.

Esempi:

$$\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline \end{array} \cdot \begin{array}{|c|} \hline 2 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 2 & 4 & 6 \\ \hline \end{array}$$
$$\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline \end{array} \cdot \begin{array}{|c|} \hline 1 \\ \hline 2 \\ \hline 3 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 2 & 4 & 6 \\ \hline 3 & 6 & 9 \\ \hline \end{array}$$

7.2.1 Spostare sulla GPU

Gli attributi di un tensore sono: forma, tipo di dato e **dispositivo su cui si trova**; quest'ultimo può essere CPU o GPU. Di default, i tensori vengono creati sulla CPU. Se disponibile, vorremmo sfruttare il backend più efficiente dato dalla GPU. Vogliamo spostare sulla GPU i tensori in uso.

Per vedere se una GPU è disponibile ed eventualmente i parametri della GPU in uso:

```
59 torch.cuda.is_available() # Check CUDA availability
60 torch.cuda.current_device() # Current CUDA device
61 torch.cuda.get_device_name(0) # Name of the device
```

Dopo aver controllato che un device sia disponibile, si possono muovere i tensori su di esso

- in fase di creazione, tramite il parametro `device`
- tramite il metodo `.to(dst)`, dove `dst` è il dispositivo di destinazione

Esempi:


```

62 cuda = torch.device('cuda:0')
63 # Per crearlo sulla GPU
64 t1 = torch.tensor([1, 2, 3], device=cuda)
65 # Per spostarlo
66 t2 = torch.tensor([1, 2, 3])
67 t_g = t2.to(cuda)

```

7.3 Linear Neural Networks

Regressione: La regressione lineare vuole modellare il rapporto tra una variabile dipendente y (output) e una (o più) variabile (/i) indipendenti x (input). Le assunzioni sono:

- relazione lineare tra x e y
- il rumore segue una distribuzione Gaussiana

Loss function: L'obiettivo della regressione è minimizzare una funzione di errore, la quale quantifica la distanza tra valore reale $y^{(i)}$ e valore predetto dalla regressione ottenuta $\hat{y}^{(i)}$.

Squared loss: l'errore quadratico si può calcolare su una singola entry i come:

$$\ell^{(i)}(\mathbf{w}, b) = \frac{1}{2} (\hat{y}_i - y_i)^2$$

Mentre su tutto il dataset:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \ell^{(i)}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\mathbf{w}^T x_i - y_i)^2$$

7.3.1 Discesa del gradiente

Si vuole ottimizzare la loss function rispetto ai parametri, utilizzando l'algoritmo di **gradient descent**. L'idea è di calcolare il rate di cambiamento della loss function rispetto a ogni parametro e modificare ciascuno di questi nella direzione che porta a ridurre la loss il più possibile.

Bisogna decidere un δ che determina "quanto lontano guardare" sulla loss function per determinare la direzione in cui decresce di più. Su pesi e bias si "guarda" $\pm\delta$ per guardare dove la funzione decresce maggiormente.

Se i parametri variano troppo in fretta rispetto alla loss function potrebbe essere difficile capire in che direzione l'errore si riduce maggiormente.

Va determinato anche un learning rate η che determina di quanto variare i parametri al termine dell'iterazione, sempre secondo la direzione determinata precedentemente.

Se vuoi sapere come si calcola il gradiente.

Si vuole calcolare la derivata della loss rispetto a un parametro, per fare ciò si può applicare la chain rule e calcolare la **derivata della loss rispetto al suo input moltiplicato la derivata del modello m** (si intende il modello della funzione usata per fare

la regressione, in questo caso lineare, quindi $wx + b$) rispetto al parametro

$$\nabla_{w,b} = \left(\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b} \right) = \left(\frac{\partial L}{\partial m} \cdot \frac{\partial m}{\partial w}, \frac{\partial L}{\partial m} \cdot \frac{\partial m}{\partial b} \right)$$

Dopo aver inizializzato i parametri, li si aggiorna finché $\mathbf{w}$ e b non cambiano più (variazione $\leq \epsilon$), oppure si fissa un numero massimo di iterazioni (ci sono altre termination conditions possibili).

Se l'aggiornamento dei parametri è troppo grande, si può avere un'oscillazione eccessiva del modello e conseguente divergenza del training.

Normalizzazione: I gradienti di matrice dei pesi e bias sono solitamente su ordini di grandezza diversi, rendendo un solo learning rate inefficace.

Al posto che usare più learning rate diversi, si **normalizzano i valori all'interno di un range**.

Sequential learning: Al posto che processare l'intero data set assieme, si può considerare un punto per volta, aggiornando i parametri dopo ognuno (**online learning**).

Esempio di algoritmo di apprendimento sequenziale è **stochastic (sequential) gradient descent**.

I modelli vengono addestrati facendo *training*: con questo termine si intende un'ottimizzazione iterativa dell'algoritmo che aggiorna i parametri in una direzione che decresce la error function man mano.

Algoritmo gradient descent: Un possibile algoritmo:

1. Inizializzazione random dei pesi, scegliere un learning rate η
2. Fino a convergenza ripetere:
 - Calcolare il gradiente

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}$$

- Aggiornare i pesi

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}$$

3. Ritorna i valori $\mathbf{w}$

Problema: calcolare il gradiente di L su tutto il training set per un singolo update è troppo costoso.

Minibatch Stochastic Gradient Descent (SGD): Come prima, ma con piccoli batch:

1. Inizializza casualmente i valori
2. Ripetere fino a convergenza:

- Scegliere un minibatch β (subset degli esempi di training)
- Calcolare il gradiente di L ristretto su β :

$$\partial_{\mathbf{w},b}\ell^{(i)}(\mathbf{w},b)$$

- Aggiornare i parametri

$$(\mathbf{w},b) \leftarrow (\mathbf{w},b) - \frac{\eta}{|\beta|} \sum_{i \in \beta} \partial_{\mathbf{w},b}\ell^{(i)}(\mathbf{w},b)$$

Come scegliamo learning rate η e dimensione del minibatch $|\beta|$? Possono essere specificati manualmente, con poi eventuale tuning.

In generale, non possono essere troppo piccoli (troppo tempo, workload troppo piccolo) o troppo grandi (oscillazioni, problemi di memoria).

7.3.2 Autograd

Autograd di PyTorch è un componente che automatizza il calcolo della discesa del gradiente. Permette backpropagation automatica, ogni tensore "ricorda" lo storico delle sue operazioni; calcolare manualmente le derivate è inutile.

Per applicare autograd, è necessario riscrivere modello e loss function usando `requires_grad=True` e chiamare `.backward()` on the loss.

Ogni tensore derivato da uno con `requires_grad=True` manterrà uno storico delle sue computazioni. Si può aggiungere il parametro `requires_grad=True` alla creazione del tensore.

Dopo aver chiamato `.backward()`, PyTorch calcola automaticamente il gradiente in `.grad` (se differenziabile).

Esempio:

```
68 loss = loss_function(model(t_u, *params), t_c)
69 loss.backward()
70 print(params.grad)
71 # tensor([4517.2969, 82.6000])
```

Altri optimizer: Oltre al classico gradient descent, si possono utilizzare altri modelli di ottimizzazione, tramite `torch.optim`.

Ogni optimizer PyTorch espone due metodi core:

- `zero_grad()`: elimina tutti i `.grad` gestiti dall'optimizer
- `step()`: aggiorna i parametri dei valori basandosi sull'algoritmo scelto

Esempio:

```
72 optimizer = torch.optim.SGD(params, lr=learning_rate)
```

7.4 Multi Layer Perceptron MLP

Reti neurali con un solo layer sono limitati, usiamo i **Multilayer Perceptrons**, per permettere più *espressività* al modello (non solo lineare). Dai che lo sai come funzionano, non li voglio spiegare.

Le cose importanti sono:

- Struttura: ingresso, hidden layers, uscita
- I neuroni hanno funzioni di attivazione, solitamente non lineari, possono essere di diverso tipo
- Il training funziona tipo:

Algorithm 1 Training Process

```
1: for  $n$  epoche do
2:   dividi dataset in batch
3:   for every batch do
4:     for every sample do
5:       valutazione del modello
6:       calcolo della loss
7:       gradient backpropagation
8:     end for
9:     update del modello
10:  end for
11: end for
```

7.4.1 Modulo nn

PyTorch fornisce un modulo **nn** dedicato alla costruzione di reti neurali.

Possiamo mantenere invariata la loss function ma ridefinire il modello usando semplici reti neurali. Il nuovo modello include:

- un modulo lineare
- una funzione di attivazione, generalmente non lineare (hidden layer)
- un altro modulo lineare per produrre l'output

Anche se input e output sono entrambi scalari, generalmente l'hidden layer ha più di una unità, fornendo maggiori capacità al modello.

Il layer finale combina le attivazioni linearmente per produrre l'output finale.

Il modulo **nn** fornisce una maniera semplice per concatenare moduli tramite il container **nn.Sequential**. Esempio:

```
73 seq_model = nn.Sequential(  
74     nn.Linear(1, 13),  
75     nn.Tanh(),  
76     nn.Linear(13, 1))
```

Si può quindi iterare il processo di training della rete.